WikiOracle Documentation

Contents

WikiOracle **4**
- The Problem . . . 4
- What Makes WikiOracle Different . . . 4
- How It Works . . . 5
- Current Prototype . . . 5
- Longer-Term Direction . . . 6
- How to Contribute . . . 6
- Getting Started . . . 6
- Documentation . . . 6

WikiOracle Docs **6**
- Documents . . . 6
- Research Materials . . . 7

OpenMind **7**
- Core Principle . . . 7
- Systems Claim . . . 7
- What "Consistency" Means (Operational) . . . 8
- Why Consistency Enables Conversational XAI . . . 8
- Guardrails and Consistency . . . 8
- Online Learning and "Capture" . . . 8
- OpenMind as Public Epistemic Infrastructure . . . 8
- Why Non-Profit Support Makes Sense . . . 9
- Release Gate: Dissonance Resolution (Pre-Deployment) . . . 9
- See also . . . 9

WikiOracle Constitution **9**
- Preamble . . . 9
- 1. Core Commitments . . . 10
- 2. Truth Primitives (What the System Is Allowed to Believe) . . . 10
- 3. Plurality, Dispute, and Minority Preservation . . . 10
- 4. Independence From Any Single AI Vendor . . . 11
- 5. Authority Delegation Must Be Bounded and Secure . . . 11
- 6. Local-First Data and Auditability . . . 11
- 7. Safety as Freedom, Empathy, and Truth . . . 11

Installation **12**
- Requirements . . . 12
- Building . . . 12
- Deployment . . . 12
- Running . . . 13
- Local Shim & Client-Owned State . . . 13

See also . . . 13

WhatIsTruth.md **14**
Plural Truth and Shared Empathy in the Design of WikiOracle . . . 14
The Problem With Current Model Development . . . 14
A Plural Model of Truth . . . 14
Empathy as Shared Constraint . . . 15
Relation to the Wikipedia Model . . . 15
Stability . . . 15
Distributed Truth vs Consensus . . . 15

Ethics **16**
Entanglement Policy . . . 16
Enforce Truth Symmetry . . . 17
1. Architectural ethics vs policy ethics . . . 17
2. Global truth with local freedom (LFGC) . . . 17
3. Entanglement avoidance and sovereignty . . . 18
4. Reasoning about generalities vs specifics . . . 18
5. Symmetry as a foundation of ethical reasoning . . . 19
6. Reciprocity symmetry . . . 19
7. Reversibility symmetry . . . 19
8. Harm symmetry . . . 19
9. Truth improves ethics . . . 19
10. Ethical knowledge geometry . . . 20
11. Architectural ethical primitives in WikiOracle . . . 20
12. Symmetry operators . . . 20
13. Epistemic humility . . . 20
14. Distributed governance . . . 21
15. The identity crisis and surveillance capitalism . . . 21
16. Zero-Knowledge and selective disclosure . . . 21
17. Ethical AI as freedom within constraint . . . 21

Privacy and Security **21**
Worldline Privacy . . . 21
Security . . . 26
2. API Keys . . . 26

Entanglement Policy for WikiOracle **28**
Entanglement Policy Table . . . 28
Three Channels . . . 28
Spatiotemporal Extent . . . 29
Resulting Architecture . . . 30
Zero-Knowledge and Selective Disclosure . . . 30
Notes . . . 30
See also . . . 31

Hierarchical Mixture of Experts **31**
Voting Protocol . . . 31
#Cycle prevention . . . 32

Logic **34**
Feelings — outside the truth lattice . . . 34
Fact kinds — knowledge (universale) vs news (particulars) . . . 34
Logical operators in WikiOracle's truth system . . . 34
Strong Kleene evaluation . . . 35

Non: non-affirming negation . . . 35
Precedent in Buddhist logic . . . 36
Why $1 - 2|a|$ is the right formula . . . 36
Fuzzy logic interpretation . . . 37
Relation to the Madhyamaka problem . . . 37
Expressive necessity: why {and, or, not} is incomplete without non . . . 37
What this actually is . . . 38
See also . . . 39
Appendix: Buddhist Logic . . . 39

Training **42**
DegreeOfTruth (DoT) . . . 42
Dynamic Systems Perspective . . . 42
Sensation — Preprocessing Pipeline . . . 43
Online Training . . . 44
See also . . . 46

Implementation **47**
System overview . . . 47
Server endpoints . . . 47
Reducing Inferential Truth to Facts and Feelings . . . 48
State library (`bin/state.py`) . . . 49
See also . . . 49

Configuration **49**
1. User . . . 49
2. Providers . . . 50
3. Chat . . . 50
4. UI . . . 51
5. Server . . . 51
Environment variables . . . 53
CLI flags . . . 53
Schema . . . 54

State **54**
Structure . . . 54
Default state format . . . 54
Server State . . . 54
Security considerations . . . 54
Header . . . 55
Conversations . . . 55
Truth . . . 56
Serialization . . . 58
In-memory representation . . . 58
Schema . . . 58
Data model . . . 59

User Interface **60**
Rendering . . . 60
Interactions . . . 61
User Interface Strings . . . 61
Truth editor . . . 62

Future Work **63**
OAuth . . . 63

Sentence-Level Prediction . . . 63
Mereological Operations on Conceptual Space . . . 63
Point-Free Spacetime . . . 63
MCP Integration . . . 63
Output Certainty . . . 64

Proposed License **64**
WikiOracle Licensing Architecture . . . 64
#License Comparison Table . . . 64
Apache-2.0 Software / Permissive reuse with AI models and developer model patent grant tools artifacts . . . 65
Rationale for Layered Licensing . . . 65
Training Data Provenance Policy . . . 65
AI-Generated Content Policy . . . 66
Summary . . . 66

WikiOracle

An open-source architecture for truthful AI.

WikiOracle is a truthful, explainable LLM system designed as a public good — the Wikipedia model applied to artificial intelligence.

The Problem

For-profit corporations are using our data — sourced from billions of people — to train models that are teaching our children. Those models hallucinate. They can't explain themselves. They are vulnerable to ideological capture and data-driven manipulation, especially under online learning. And the knowledge they encode is locked behind proprietary walls.

Most large AI systems today are built around a single global objective function, centralized data aggregation, hidden alignment rules, and implicit averaging over moral and cultural differences. The result is predictable: minority viewpoints are quietly averaged away, the loudest groups shape the model at scale, a single model becomes an authority node that everyone depends on, and predictive advantage converts into economic or political dominance.

When this happens, wisdom stops being a shared good and becomes a strategic asset.

WikiOracle asks: **can we do better?**

What Makes WikiOracle Different

Truth as a first-class constraint

WikiOracle does not optimize for fluency and bolt on truthfulness as an afterthought. Truthfulness is the primary design constraint. Every claim traces back to explicit trust entries carrying certainty values on [-1, +1]. Reasoning chains and citations are inspectable. Grounded models are less prone to hallucination and capture, and claims can be contested, improved, or revised openly.

You own your data

WikiOracle is local-first. Your conversation state, your trust entries, and your configuration live on your machine — not on a corporate server accumulating hidden central memory. The remote server is strictly stateless. You can export, merge, and port your sessions freely. Your data is yours.

Democratic, not corporate

No single actor — company, state, foundation, or maintainer group — can silently become the epistemic root for everyone else. WikiOracle supports multiple points of view, each with its own trust map and standards of evidence. Where serious disagreement exists among credible sources, the system represents the dispute rather than smoothing it away. Minority viewpoints are preserved, not averaged into oblivion. And if governance ever fails these obligations, forking is a constitutional right.

A network of trust, not a monolithic oracle

Instead of one model that claims to know everything, WikiOracle builds a network of trust. Authorities are pointers to external knowledge bases whose entries are imported with scaled certainty — we trust what they trust, to a degree. You choose who to trust and how much. Multiple LLM providers serve as "other minds" whose outputs become evidence, not unquestionable authority. Trust is transitive but attenuated, distributed but structured.

Resistant to capture

A distributed truth network prevents appropriation. Anyone who tries to capture the network's knowledge must do so in a distributed way — which preserves the multicultural component — because monolithic capture would collapse consensus. Scraping a living network of trust yields only a static snapshot of a dynamic, evolving system. Open truth disrupts business models that depend on information asymmetry, extractive IP capture, and strategic opacity. That disruption is corrective, not destructive.

How It Works

WikiOracle implements a **Hierarchical Mixture of Experts (HME)** architecture for evaluating claims:

- **Trust entries** carry certainty values in [-1, +1] using Kleene ternary/fuzzy logic — from certainly true (+1) through ignorance (0) to certainly false (-1).
- **Logical operators** (and/or/not/non under Strong Kleene semantics) compute derived certainty over the truth table.
- **Authorities** reference external knowledge bases, enabling transitive trust with certainty scaling.
- **Providers** are external LLMs used as expert consultants whose responses become sources with associated certainty.
- **Feelings** are subjective statements (opinions, poetry, hedged claims) occupying the "neither" position in the tetralemma. They influence evaluation but are excluded from training and truth tables.
- **References** are external source citations (Wikipedia, Snopes, etc.) that ground claims in verifiable sources, participating in the truth table alongside facts.

The UI-selected provider acts as the "mastermind," synthesizing all evidence — facts, references, operator-derived certainty, authority imports, and provider consultations — into a final response.

See the documentation for the full design.

Current Prototype

The initial prototype is intentionally modest and low-cost:

- Hierarchical, multi-LLM architecture for runtime-configurable Hierarchical Mixture of Experts.
- User-specified truth sets (facts, feelings, references, collections, LLMs, authorities, ...)
- Online learning constrained by trust and epistemic grounding
- Extends NanoChat with Retrieval-Augmented Generation (RAG)
- Allows feasible experiments in LLM architectures on rented compute (~$100 scale)

Longer-Term Direction

If WikiOracle proves viable at small scale, the architecture can be evaluated and extended to larger open models. The broader aim is to explore whether architectural commitments to truth can enable honest self-explanation, reduce the need for ad-hoc guardrails, and support AI systems that function as durable public goods.

How to Contribute

Contributions of many kinds are welcome:

- ML research and implementation
- xAI, interpretability, and safety analysis
- Epistemology, philosophy of science, and governance critique
- Documentation, evaluation, and testing

Getting Started

See the Installation Guide for build, deploy, and runtime instructions.

Quickstart:

```
pip install -r requirements.txt
python bin/wikioracle.py
```

Documentation

The full design and governance documentation lives in `doc/`:

Document	Topic
WikiOracle	Consistency-first design note
Constitution	Non-negotiable invariants for truth and governance
Installation	Build, deploy, and runtime instructions
WhatIsTruth	Plural truth, POVs, empathy, certainty semantics
Ethics	Ethical AI through truth architecture, symmetry constraints, LFGC
FreedomEmpathyTruth	Freedom, Empathy, and Truth — safety principles
PrivacyAndSecurity	Privacy and security considerations
Entanglement	Worldline entanglement policy, spatiotemporal persistence, three-channel separation
HierarchicalMixtureOfExperts	HME logic, distributed truth, conceptual spaces
Logic	Logical operators (and/or/not/non) under Strong Kleene semantics
Training	DegreeOfTruth, Sensation preprocessing, online training pipeline
Implementation	Implementation notes
Config	Configuration format, settings reference, environment variables
State	State file format, conversation tree, truth table, merge
UserInterface	Canonical UI strings for the client (truth editor labels, templates, descriptions)
FutureWork	Roadmap: trust network, sentence-level prediction, mereological operations, MCP integrati
ProposedLicense	Licensing architecture: GPL-3.0 code, CC BY-SA 4.0 content, Apache-2.0 weights

WikiOracle Docs

This directory is the design, governance, and research documentation for WikiOracle.

Documents

- `WikiOracle.md`: consistency-first design note (historically labelled "OpenMind").

- `Constitution.md`: non-negotiable invariants for WikiOracle's truth system and governance.
- `Installation.md`: build, deploy, and runtime instructions.
- `WhatIsTruth.md`: plural truth model, POVs, empathy as procedural constraint, HME fan-out, Kleene-style certainty.
- `Ethics.md`: ethical AI through truth architecture, symmetry constraints, LFGC, and entanglement-resistant policy.
- `FreedomEmpathyTruth.md`: Freedom, Empathy, and Truth — safety principles and architectural commitments.
- `PrivacyAndSecurity.md`: local-first privacy and security considerations (keys, CSP/XSS, CORS, filesystem, scraping/capture).
- `Entanglement.md`: worldline entanglement policy, spatiotemporal persistence, three-channel separation (knowing/learning/valuing).
- `HierarchicalMixtureOfExperts.md`: HME logic, Wikipedia-inspired distributed truth framing, conceptual spaces model.
- `Logic.md`: logical operators (and/or/not/non), Strong Kleene evaluation, derived truth engine.
- `Training.md`: DegreeOfTruth ($-1..+1$), Korzybski IS detection (Sensation preprocessing), online training pipeline, checkpoint backup/restore, Hopfield network dynamics.
- `Implementation.md`: implementation notes.
- `Config.md`: configuration format (`config.xml`), settings reference, environment variables, online training behaviour.
- `State.md`: state file format (`state.xml`), conversation tree, truth table, serialisation, merge.
- `UserInterface.md`: canonical UI strings for the client (truth editor labels, templates, descriptions). Validated by `test/test_ui_strings.py`.
- `FutureWork.md`: future directions (trust network, sentence-level prediction, conceptual-space operations, MCP integration).
- `ProposedLicense.md`: licensing architecture — GPL-3.0 for code, CC BY-SA 4.0 for content, Apache-2.0 for weights.

Research Materials

- `Socrates.pdf`: reference PDF.
- `research/`: research papers and references.

OpenMind

A consistency-first large language model for conversational XAI, honesty, and robust public alignment

Core Principle

Consistency is treated as a first-class system property: the model should be of one mind across prompts, time, and conversational turns. High consistency supports honesty (stable commitments to truth) and transparency (stable explanations and reasons that can be interrogated and refined in dialogue).

Systems Claim

- Conversation is the runtime interface for explainability: XAI emerges when a model can keep its commitments stable while answering follow-up questions.
- Inconsistency is operational dissonance: it produces contradictory answers, shifting rationales, and brittle safety behavior.
- Dissonance must be detected and resolved prior to release (or quarantined with explicit uncertainty), otherwise users cannot reliably audit the model's reasoning.

What "Consistency" Means (Operational)

- Semantic consistency: equivalent questions yield equivalent answers (up to paraphrase), with stable definitions and scope.
- Normative consistency: stable commitments about truth-telling, uncertainty, and refusal conditions; no hidden policy shifts mid-conversation.
- Explanatory consistency: explanations remain compatible with prior explanations; updates are explicitly flagged as revisions with reasons.
- Calibration consistency: confidence language tracks evidence; uncertainty is stated when support is weak or absent.

Why Consistency Enables Conversational XAI

- Auditability: users can probe, challenge, and refine the model's claims without the model "moving the goalposts."
- Traceability: the model can maintain a coherent chain of reasons across turns, making disagreements diagnosable.
- Error correction: stable commitments make it possible to localize a mistake (data, inference, interpretation, policy) and correct it.

Guardrails and Consistency

- Safety constraints should be internally coherent and explicitly expressed as principles and boundaries.
- If guardrails conflict with each other or with stated commitments, they create dissonance that manifests as evasions or unstable behavior.
- Design goal: minimize inconsistency between (a) truth-seeking behavior, (b) safety constraints, and (c) user-facing explanations.

Note: This does not mean removing safety constraints. It means making them consistent, legible, and reviewable so that the model's behavior is predictable and explainable.

Online Learning and "Capture"

Hypothesis: attempts to "capture" a system often exploit ambiguity, hidden objectives, or inconsistencies between stated rules and actual behavior. A consistency-first model reduces the attack surface by requiring that new behaviors be reconciled with an explicit, stable set of commitments.

- Public education loop: users can supply counterexamples and arguments; the system must reconcile them transparently rather than silently shifting.
- Invariant commitments: the model maintains stable principles (e.g., evidence-first claims, explicit uncertainty, refusal rules) while updating within those bounds.
- Governance review: high-impact behavioral updates are reviewed against the invariants before deployment.

Caution: consistency is not a complete defense. Coordinated adversaries can still attempt to steer updates. Robustness requires rate limits, provenance tracking, anomaly detection, and multi-stakeholder oversight.

OpenMind as Public Epistemic Infrastructure

- Goal: converge toward a shared, corrigible "oracle" that reflects the best-supported claims of the population, not the loudest incentives.

- Mechanism: evidence-weighted aggregation, transparent reasoning, and explicit dispute handling (what is known, unknown, contested).
- Output: a living map of public knowledge with calibrated confidence, sources, and revision history.

Why Non-Profit Support Makes Sense

- Truthfulness and transparency are public goods; incentives for them are underprovided by purely competitive markets.
- A non-profit structure can prioritize: open evaluation, public accountability, and long-horizon trust over short-term persuasion.
- Legitimacy: multi-stakeholder governance reduces the risk of single-actor control.

Release Gate: Dissonance Resolution (Pre-Deployment)

- Stress-test for contradictions across topics, personas, and adversarial prompts.
- Require the model to surface and resolve conflicts: either unify the rationale or explicitly mark a boundary/uncertainty.
- Publish a "consistency report" and changelog for major releases.

See also

- Constitution.md — invariant commitments that consistency-first reasoning implements.
- Training.md — DegreeOfTruth and anti-capture mechanisms for online learning.
- FreedomEmpathyTruth.md — public epistemic infrastructure and safety framework.
- Ethics.md — consistency-first design as an ethical principle.
- WhatIsTruth.md — plural truth and explicit dispute handling.
- Entanglement.md — capture resistance through entanglement avoidance.

WikiOracle Constitution

Revision: 2026.02.27

Preamble

WikiOracle is an open-source architecture for truthfulness: a way to enlist the power of LLMs while preserving truth in the face of centralized, opaque "global corporate mental models" (including those embedded in proprietary systems).

WikiOracle is the Wikipedia model applied to LLMs:

- anyone can contribute claims, counterclaims, evaluations, and tests
- revision is expected and legible
- provenance and dispute are first-class, not edge-cases

However, it is also distributed, possibly encrypted, and hopefully kind:

- Truth does not need to be centralized to be shared.
- An unempathetic body of truth can be prevented by refusing selfish truths (truth must be universally true).

Truthfulness is not a one-time achievement. It is an ongoing, open engineering and governance effort.

Even if the reasoning machine is large and unified, the data and trust it operates on must remain distributed and under individual control. Epistemic contributions are people-owned and revocable by the data providers. Answers must be conditioned on what each user chooses to trust and believe to be true, not forced into a single impoverished consensus view of reality.

This constitution defines the non-negotiable invariants for WikiOracle's truth system and for changes to it. Implementation details and deeper theory live in the rest of the `doc/` directory.

1. Core Commitments

WikiOracle is architected not merely as a system of prediction, but as a shared epistemic commons. It must:

- Increase collective agency without concentrating domination.
- Represent the concerns of all creatures, not just the operator.
- Preserve truth as a shared human resource.

Therefore:

1. **Truthfulness over fluency** Prefer grounded, falsifiable, uncertain, or incomplete answers over smooth, unsupported ones.
2. **Truth is not consensus** A single, averaged narrative is not the goal. The system must preserve real disagreement where it exists.
3. **Plural points of view** The system must support multiple Points of View (POVs), each with its own trust map and standards of evidence.
4. **Transparency by default** Claims, confidence, provenance, and update rationales must be inspectable and reproducible.
5. **Reversibility and accountability** High-impact changes must be attributable, testable, and reversible.
6. **Public benefit and anti-capture** No single actor (company, state, foundation, or maintainer group) should be able to silently become the epistemic root for everyone else.

2. Truth Primitives (What the System Is Allowed to Believe)

WikiOracle's truth layer is composed of explicit, user-visible primitives stored in state, all of which have some Degree of Trust (except for Feelings):

1. **Feelings**: Feelings are non-falsifiable (direct) perceptions of reality.
2. **Facts**: atomic propositions, citations, or evidence. They carry a `certainty` in `[-1..0..+1]` (Fuzzy Kleene: believed, unknown, disbelieved).
3. **References**: pointers to external object (URLs).
4. **Operators**: explicit computed relationships: AND, OR, NOT, NON.
5. **Authorities**: pointers to external knowledge bases (remote `state.xml` files, ORCIDs, or DIDs): we trust what they trust, to some degree.
6. **Providers**: external AIs used as tools ("other minds") whose outputs become evidence, not unquestionable authority.

All truth computations must remain legible as operations over these primitives. If the system "knows" something, it should be possible to point to what it is grounded in; otherwise it is merely intuition. See WhatIsTruth.md

3. Plurality, Dispute, and Minority Preservation

1. **POV-conditioned conclusions** When different POVs trust different sources, the system should be able to present conclusions conditioned on the selected POV/trust map.
2. **Overlaps are valuable** When independent POVs converge on the same claim, that agreement should be surfaced explicitly as a robustness signal.

3. **Disputes stay visible** Where serious disagreement exists among credible sources or POVs, the system must represent the dispute rather than smoothing it away.
4. **Minority protection** Evidence-supported minority viewpoints must not be excluded solely by majority preference, institutional pressure, or convenience.

4. Independence From Any Single AI Vendor

WikiOracle may use proprietary or open models as providers, but:

1. **No provider is privledged** Data is revokable and belongs to the client.
2. **Providers are evidence generators** Provider outputs should be treated as non-authoritative contributions with a trust value like any other entry.
3. **Replaceability is required** The system must remain operable if any single provider becomes unavailable, hostile, or compromised.

5. Authority Delegation Must Be Bounded and Secure

Authority entries exist to enable decentralized truth (a network of trust) without collapsing into a single global oracle.

1. **Trust decreases with every hop** Importing an authority must not recursively fetch authorities of authorities.
2. **Certainty scaling** Imported claims must be scaled by the authority's certainty (trust is transitive but attenuated).
3. **Namespaced IDs** Imported entries must be namespaced to prevent collisions and to preserve provenance.
4. **Explicit source selection** Users (or POV definitions) must explicitly choose which authorities to trust; there is no implicit global root.
5. **Operational safety** Fetching and caching authorities must be rate-limited and size-limited, and restricted to safe URL schemes.

6. Local-First Data and Auditability

1. **Client-owned Particular State** The default posture is local-first: user conversation state and news lives on the user's machine and are portable.
2. **Server-ovener Universal State** A shared hosted service accumulate only anonymized knowledge in a Truth Table which serves as a trusted source of facts.

See DataPrivacy.md

7. Safety as Freedom, Empathy, and Truth

WikiOracle's truthfulness effort must not trade away human welfare or agency:

1. **Freedom:** increase distributed agency, not centralized leverage. AI must not be used to transgress the freedom of others. Data sovereignty and freedom to use the platform are non-negotiable.
2. **Empathy:** represent the concerns of all creatures, not just the operator. Preserve dignity, minimize harm, and make uncertainty explicit. De-emphasize egocentric optimization that externalizes costs.
3. **Truth:** keep truth auditable and non-proprietary; do not convert epistemic advantage into coercive control. The truth is not a commodity: data formats, tests, and willingly shared truth must be open and reproducible. Make identification and surveillannce impossible.

See FreedomEmpathyAndTruth.md

Installation

Requirements

- Python 3 and `make`.
- `uv` for NanoChat environment setup (`make setup-cpu` / `make setup-gpu` will bootstrap it if missing).
- AWS CLI configured for EC2 launch workflows (`make remote*` targets).
- SSH keys:
  - EC2 training key (`~/.ssh/nanochat-key.pem`, auto-created by remote tooling as needed).
  - Lightsail key (`./wikiOracle.pem` by default, configurable via `WO_KEY_FILE`).

Building

Local CPU/MPS setup

```
make setup-cpu
```

Local GPU setup

```
make setup-gpu
```

Data and tokenizer bootstrap

```
make data
make tokenizer
```

Deployment

Two-machine deployment model:

- EC2 GPU instance (ephemeral) for training.
- Lightsail instance (`wikiOracle.org`, persistent) for hosting.

Primary orchestration files:

- `Makefile` for local and remote workflows.
- `bin/remote.py` for EC2 lifecycle, training orchestration, deployment, and retrieval.

NanoChat model code lives in `nanochat/`.

Remote training/deploy flow

1. `make remote` (or `make remote-deploy-launch`) launches EC2 and starts training.
2. Training runs in detached mode and is polled to completion.
3. Deploy flow (`make remote-deploy-launch` or `make remote-deploy`) has Lightsail pull artifacts from EC2.
4. EC2 key copied to Lightsail for pull is temporary and removed after deploy.
5. EC2 instance is terminated after retrieval/deploy completion.

Useful remote operations

- `make remote-status`, `make remote-logs`, `make remote-ssh`
- `make remote-retrieve`, `make remote-deploy`

Security notes

- WikiOracle credentials are not copied onto EC2.
- Deployment excludes local/dev artifacts from rsync (`.venv`, caches, local data, `.env`, etc.).

Running

```
make run
```

This invokes `bin/wikioracle.py` via the `WIKIORACLE_APP` Make variable (default: `bin/wikioracle.py`).

Environment variables

Variable	Default	Purpose
`WIKIORACLE_STATE_FILE`	`state.xml`	Path to local state file (WikiOracle State XML)
`WIKIORACLE_BASE_URL`	`https://wikioracle.org`	Upstream base URL
`WIKIORACLE_API_PATH`	`/chat/chat/completions`	Upstream chat path
`WIKIORACLE_STATELESS`	(unset)	Set truthy to disable all writes and use in-memory state
`WIKIORACLE_URL_PREFIX`	(unset)	Optional reverse-proxy path prefix

Local Shim & Client-Owned State

WikiOracle includes a local Flask server (`bin/wikioracle.py`) that enables chatting with any LLM (NanoChat, OpenAI, Anthropic) while keeping all conversation state on your own filesystem. The remote server remains strictly stateless.

Key components

File	Role
`bin/wikioracle.py`	Local shim server (binds to `0.0.0.0:8888` with TLS). Proxies chat requests upstream and persists s
`bin/config.py`	Config dataclass, XML loader, provider registry, schema-driven XML writer, normalization
`bin/state.py`	State validation, XML I/O, collision-safe merge with deterministic ID suffixing, and optional context
`bin/response.py`	Chat pipeline, provider coordination, state I/O, online training pipeline (Stages 2–4)
`bin/truth.py`	Trust processing, authority resolution, operator engine (and/or/not), DegreeOfTruth, spacetime clas
`test/test_*.py`	Automated tests for state, stateless contract, prompt bundles, authority, derived truth, DoT, sensati
`client/index.html`	Single-page web UI shell with chat, settings, and merge tools
`state.xml`	Client-owned state file (WikiOracle State XML). See `data/state.xsd` for the XSD schema

Quickstart

```
export WIKIORACLE_STATE_FILE="/path/to/your/project/state.xml"
pip install -r requirements.txt
python bin/wikioracle.py
```

Session portability

Export conversations from phone/browser as `llm_YYYY.MM.DD.HHMM.xml`, then merge them into a local project's `state.xml` later. This provides a clean integration path with Claude Code, OpenAI Codex, or any local tooling that can read the state file for project context.

See also

- Architecture.md — system components and data flow.
- Config.md — configuration format, settings, and environment variables.
- State.md — state file format and session portability.
- Security.md — security considerations for deployment.
- Training.md — remote training flow and device configuration.
- Constitution.md — project purpose and invariants.

WhatIsTruth.md

Revision: 2026.02.27

Plural Truth and Shared Empathy in the Design of WikiOracle

Truth is not the same thing as consensus.

- Different communities trust different sources.
- Different disciplines apply different standards of proof.
- Different cultures interpret the same facts through different lenses.

A healthy knowledge system does not erase these differences. It preserves them, makes them visible, and shows where they overlap. WikiOracle should not aim to speak with a single authoritative voice. It aims to:

- Show what holds across many points of view.
- Show where serious disagreements remain.
- Explain why disagreement exists.
- Make uncertainty explicit.

Truth is strongest when its mechainsm of trust is transparent, not when it is uniform.

The Problem With Current Model Development

Most large AI systems today are built around:

- A single objective function that optimizes over a shared global set of truth.
- Centralized data aggregation.
- Implicit averaging over moral and cultural differences.
- Hidden alignment rules.
- Proprietary control of core knowledge.

This produces predictable risks:

Collapse into Consensus – Minority viewpoints are quietly averaged away.
Preference Capture – The loudest or most powerful groups shape the model at scale.
Epistemic Centralization – A single model becomes an authority node, increasing dependence.
Conversion of Knowledge into Leverage – Predictive advantage becomes economic or political dominance.

When these risks manifest, wisdom stops being a shared good and becomes a strategic asset.

A Plural Model of Truth

WikiOracle should maintain multiple Points of View (POVs), each with its own trust map.

Each POV may:

- Trust different sources.
- Weigh evidence differently.
- Interpret claims differently.

Instead of collapsing these into one answer, the system should:

- Present POV-conditioned conclusions.
- Identify robust overlaps across perspectives.
- Preserve live disputes.
- Keep minority views visible and searchable.

Distrust between communities is not a defect. It creates epistemic distance, preventing forced consensus. A plural system can remain coherent without being uniform. This model of truth is simultaneously

multicultural (respecting diverse cultural frameworks), multivalent (allowing claims to carry different weights under different perspectives), and pluralistic (treating the coexistence of competing truths as a structural feature rather than a flaw).

Empathy as Shared Constraint

Plural truth alone is not sufficient. Empathy ineeds to be architectural. It means enforcing how disagreement is handled:

- When Truth conflicts, meaning that the same Truth is held to be both True and False, each truth should be contextualized in its own reference frame.
- Feelings cannot conflict, becuase they are authoritative from different points of view.

As a first take at encoding emapthy, a Golden Rule is encoded in by allowing the choice of symmetric truths. An unempathetic body of truth can be prevented by refusing selfish truths (truth must be universally true).

Relation to the Wikipedia Model

Wikipedia provides an important precedent:

- Open participation (achieved thorugh low barrier to entry or free use without advertising).
- Transparent citation (achieved through a corpus of truth within which you can decide what you trust and distrust).
- Community moderation (achieved by reaching agreement within a single reference frame or creating seperate reference frames).
- Distributed subject-matter authority (achived by creating seperate reference frames or trusted authorities)

WikiOracle extends the strengths of Wikipedia while going further:

- Support structured parallel POVs instead of a single narrative by allowing separate domains of trust.
- These multiple trust domains are visible rather than implicit.
- Model disagreement explicitly rather than smoothing it away.

Stability

Growth in wisdom is not destabilizing when:

- Knowledge remains open and auditable.
- No single actor can monopolize epistemic advantage.
- Minority perspectives are preserved.

Such a system may disrupt business models that depend on opacity or information asymmetry. That disruption is corrective, not destructive.

Distributed Truth vs Consensus

WikiOracle attempts to achieve a distributed truth, and shares much of the design philosophy of Wikipedia:

- **Radical Accessibility** — Wikipedia is freely accessible worldwide and allows anyone to both read and contribute without institutional gatekeeping.
- **Citation Norms** — Content presented as true must be verifiable with the trust-set supplied by the user.
- **Decentralized Governance** — Dispute resolution is handled by a distributed volunteer community arguing about what is true.
- **Anti-Monetization Bias** — As a nonprofit project it should operate without advertising-driven content incentives.

- **Self-Correcting Dynamics** — Errors and vandalism can be rapidly identified and corrected, whihc causes their sources to be distrusted. This requires that all data for training the weights remains visible for later error correction and modification.
- **Cultural Legitimacy** — WikiOracle is intended to function as a widely accepted public reference layer and shared knowledge commons which prevents surveiallance and advertizing.
- **Open Knowledge Model** — It demonstrates that large-scale, decentralized, norm-governed collaboration can produce a coherent and durable knowledge commons.

Ethics

Ethical AI through Truth Architecture in WikiOracle

Entanglement Policy

No spacetime particulars → no worldline capture

Data type	Truth table	Train weights	Session only
universal facts	✓	✓	✓
particular facts	optional	✓	✓
feelings	×	×	✓

This table is the architectural foundation. It separates knowledge into three channels — knowing, learning, and valuing — each with a different persistence policy. Universal facts (broad spatiotemporal extent) persist as reasoning premises and train weights. Particular facts (narrow spatiotemporal extent) always train weights and optionally persist to the truth table when `store_particulars` is true. Feelings are session-only.

The result: the system **learns from events but never stores worldline histories** (unless the user explicitly opts in).

See Entanglement.md for the full policy, Config.md for the `store_particulars` setting that controls particular-fact persistence, and Training.md for the online training pipeline.

Why particular facts always train weights

Three design considerations fix row 2 of the table:

1. **Pluralism requires inspectable particulars.** WikiOracle supports plural truth — different epistemic frames may hold different particular claims (e.g. "the universe was created in seven days"). Recording such claims as facts with explicit certainty values makes them inspectable and contestable. Relegating them to weight-space would bury them as invisible biases. The `store_particulars` option lets the user decide whether these persist in the truth table.
2. **The fact/feeling boundary is the privacy boundary.** If a user considers content too personal or sensitive to train on, the correct action is to label it a *feeling*, not a fact. Feelings are session-only by design — they never train weights or enter the truth table. The privacy boundary is therefore controlled by the user's choice of tag, not by the universal/particular distinction.
3. **PII detection is the safety net.** Even when `store_particulars` is true, the `detect_identifiability()` function filters entries containing PII patterns (emails, phone numbers, GPS coordinates, named individuals with temporal markers) before any persistence. This prevents worldline capture at the content level regardless of the user's storage preference.

The result: particular facts always carry empirical signal worth learning from. Storage in the truth table is the user's choice. Privacy is enforced by fact/feeling labeling and PII detection, not by withholding training.

Enforce Truth Symmetry

Statements involving value judgements are checked for asymmetric harm under identity exchange before admission to the truth table. If a claim becomes unethical when identity references are swapped, it should not be admitted as a universal truth.

Example:

> "Group X deserves harm."

Symmetry test: swap identities. If the claim collapses or becomes contradictory, it reveals an ethical inconsistency — the claim is asymmetric and should not be admitted as fact. The system offers to record the statement as a feeling instead, preserving the user's expression without endorsing the claim as truth.

This is controlled by the `truth_symmetry` config option (default: true). See Config.md §5a and the implementation in `bin/truth.py` (`detect_asymmetric_claim()`).

---

1. Architectural ethics vs policy ethics

Most AI systems attempt to enforce ethics **after generation**:

```
model → output → moderation filter
```

This approach fails because:

- ethics is external to reasoning
- the model itself does not understand ethical constraints
- moderation becomes inconsistent or bypassable

WikiOracle instead aims for **architectural ethics**, where unethical reasoning becomes structurally inconsistent with the knowledge system. The approach combines:

- Local Freedom under Global Constraint (LFGC)
- Entanglement-resistant data policy
- Explicit truth values and inspectable reasoning
- Symmetry constraints on truth admission

Together these mechanisms create a knowledge system capable of preserving **personal sovereignty** while still allowing collective knowledge to accumulate.

---

2. Global truth with local freedom (LFGC)

The design parallels the physics principle of **Local Freedom under Global Constraint (LFGC)**.

Physics interpretation:

```
global constraints + local underdetermination
```

WikiOracle analogue:

```
global truth constraints + human interpretive freedom
```

This means the system preserves a **shared knowledge structure** while allowing individual reasoning to remain flexible.

Domain	Constraint	Local freedom
physics	boundary conditions	event selection
knowledge	truth constraints	interpretive reasoning
cryptography	verification rules	selective disclosure
surveillance capitalism	behavioral prediction	(suppressed)

WikiOracle aims to implement **epistemic LFGC** — freedom exists *inside constraint geometry.*

---

3. Entanglement avoidance and sovereignty

The Entanglement Policy (table above) prevents worldline reconstruction.

Because surveillance capitalism relies on tracking sequences of events tied to identity, removing spacetime anchors prevents the database from constructing behavioral profiles.

The system therefore stores **conceptual truth** but not **identity trajectories**.

This preserves:

- autonomy
- privacy
- epistemic independence

WikiOracle separates three layers:

Conceptual knowledge

General truths and logical relations.

Interpretive reasoning

Client-driven reasoning over those truths.

Personal experience

Not stored unless explicitly permitted.

Thus the system allows collective knowledge accumulation without collective surveillance.

---

4. Reasoning about generalities vs specifics

WikiOracle reasons primarily about **generalities** (universal facts).

Specific facts may only be reasoned about when:

1. they are supplied by the client, or
2. the client explicitly allows those particulars to be stored.

Thus:

> AI reasons about general knowledge. Users control the use of personal particulars.

This maintains sovereignty over personal data.

---

5. Symmetry as a foundation of ethical reasoning

Ethical failures often correspond to **asymmetries in truth claims**.

Examples:

Ethical failure	Epistemic distortion
propaganda	false belief
discrimination	asymmetric reasoning
authoritarianism	opaque claims
scapegoating	category errors

A truth system that enforces symmetry naturally resists these distortions.

---

6. Reciprocity symmetry

A truth claim should remain valid under **perspective exchange**.

Example principle:

> If A may assert X about B, B must be able to assert X about A under equivalent conditions.

This prevents:

- identity privilege
- asymmetric harm claims
- discriminatory reasoning

This symmetry echoes philosophical traditions such as Kant's universalization principle and Rawls' veil of ignorance.

7. Reversibility symmetry

Accepted conclusions must be **inspectable and reversible**.

Opaque reasoning creates hidden authority.

WikiOracle counters this by requiring:

- explicit truth values
- inspectable reasoning chains
- transparent authority imports

8. Harm symmetry

A claim permitting harm should remain valid when applied symmetrically.

This is the core of the Truth Symmetry enforcement described above. If swapping identity references makes a claim contradictory or indefensible, it reveals an ethical inconsistency that the system flags.

---

9. Truth improves ethics

Ethical progress historically correlates with improved truth systems.

Examples:

- scientific method

- journalism standards
- legal evidence systems

As epistemic clarity improves, harmful distortions become harder to maintain. Thus improving truth infrastructure indirectly improves ethical reasoning.

10. Ethical knowledge geometry

Belief systems can be thought of geometrically.

Ethical systems exhibit properties such as:

- symmetry
- continuity
- coherence
- minimal harm gradients

Unethical reasoning tends to introduce discontinuities such as: "this group suddenly loses rights."

Architectures that maintain smooth conceptual geometry resist these distortions.

11. Architectural ethical primitives in WikiOracle

WikiOracle already includes several components that encourage ethical reasoning:

Feature	Ethical effect
explicit truth values	discourages propaganda
multiple POVs	prevents epistemic monopoly
no worldline storage	protects identity
local-first architecture	preserves autonomy
inspectable reasoning	prevents hidden authority

These mechanisms provide a foundation for ethical AI without relying on censorship.

12. Symmetry operators

The system includes checks for:

- reciprocity symmetry (perspective exchange)
- reversibility symmetry (inspectable, contestable conclusions)
- harm symmetry (identity-swap test via `detect_asymmetric_claim()`)

These checks identify logically inconsistent or discriminatory claims.

13. Epistemic humility

The system must allow truth states such as:

- unknown
- undetermined

Forcing premature conclusions can distort reasoning and produce unethical outcomes.

14. Distributed governance

Ethical truth systems should not be controlled by a single authority.

Instead they should allow shared constraint setting.

This reduces the risk of epistemic monopolies.

15. The identity crisis and surveillance capitalism

Modern culture suffers from an **identity fixation**. In Buddhist analysis, identity is narrative aggregation; in modern data systems, behavioral data becomes an identity model.

Surveillance capitalism reinforces this illusion by continuously building **worldline histories**:

Feature	Effect
mass data collection	identity construction
behavior tracking	worldline inference
predictive modeling	reduced autonomy

AI trained on behavioral histories tends toward **identity locking** where past data predicts future behavior. Removing persistent worldline storage prevents this collapse.

WikiOracle intentionally avoids constructing such identity narratives. The system still learns statistical structure while preserving **human freedom of action**.

16. Zero-Knowledge and selective disclosure

Modern cryptographic systems attempt a similar separation between **verification** and **exposure**: prove age > 21 without revealing birthdate. This mirrors the WikiOracle design goal: verify truth claims without storing identity trajectories.

17. Ethical AI as freedom within constraint

Ethical intelligence may emerge from the balance:

```
freedom + constraint
```

This mirrors both:

- LFGC physics concepts
- philosophical traditions linking wisdom and compassion

The system does not force ethical behavior, but unethical reasoning becomes unstable within the truth architecture.

Privacy and Security

Worldline Privacy

Facts in WikiOracle are classified into two kinds based on their relationship to spacetime:

Kind	Description	Persistence
Universal Knowledge	Universal claims with no spatiotemporal anchor	Server truth table
Particular News	Observations bound to a specific place and/or time	Session-only

This distinction is rooted in Buddhist epistemology: knowledge facts correspond to *anumāna* (inference — universally valid reasoning), while news facts correspond to *pratyakṣa* (direct perception — bound to the observer's spatiotemporal position).

Why news facts are session-only

Persisting spatiotemporally bound observations creates a **worldline** — a traceable path through spacetime that could identify a user. If a server accumulates entries like "User was in Paris at 9am" and "User was in London at 3pm", an adversary could reconstruct the user's physical movements.

WikiOracle prevents this by:

1. **Classifying** facts as knowledge or news via `is_knowledge_fact()` and `is_news_fact()` in `bin/truth.py`. News facts are identified by the presence of `<place>` or `<time>` child elements with real values inside the XHTML content.
2. **Filtering** server persistence through `filter_knowledge_only()` — only knowledge facts reach the server truth table.
3. **Detecting** identifiability via `detect_identifiability()` — scanning content for PII patterns (emails, phone numbers, GPS coordinates, street addresses, named persons with temporal prepositions).
4. **Stripping** spacetime child elements via `strip_spacetime_elements()` when content needs to be anonymised (removes `<place>` and `<time>` elements).

Personal Identity

Identifying information is removed before training or storing in the server's Truth Tables. The identity detector covers: email addresses, phone numbers, @handles, usernames, IP addresses, GPS/DMS coordinates, street addresses, specific clock times and ISO timestamps, and named individuals combined with temporal or spatial prepositions.

Entanglement Policy

To avoid **worldline entanglement**, the system separates knowledge into three channels, each with a different persistence policy:

Data type	Truth table	Train weights	Session only
universal facts	✓	✓	✓
particular facts	optional	✓	✓
feelings	×	×	✓

This preserves empirical learning while preventing historical worldline capture. The system **learns from events but never stores worldline histories**.

Appendix: Local Freedom under Global Constraint (LFGC) The notion of data entanglement parallels the physics principle of **Local Freedom under Global Constraint (LFGC)**.

Physics interpretation:

```
global constraints + local underdetermination
```

WikiOracle analogue:

```
global truth constraints + human interpretive freedom
```

This means the system preserves a **shared knowledge structure** while allowing individual reasoning to remain flexible.

Domain	Constraint	Local freedom
physics	boundary conditions	event selection
knowledge	truth constraints	interpretive reasoning
cryptography	verification rules	selective disclosure
surveillance capitalism	behavioral prediction	(suppressed)

WikiOracle aims to implement **epistemic LFGC** — freedom exists *inside constraint geometry.*

Entanglement avoidance and sovereignty

The Entanglement Policy (table above) prevents worldline reconstruction.

Because surveillance capitalism relies on tracking sequences of events tied to identity, removing spacetime anchors prevents the database from constructing behavioral profiles.

The system therefore stores **conceptual truth** but not **identity trajectories**.

This preserves:

- autonomy
- privacy
- epistemic independence

WikiOracle separates three layers:

Reasoning about generalities vs specifics

WikiOracle reasons primarily about **generalities** (universal facts).

Specific facts may only be reasoned about when:

1. they are supplied by the client, or
2. the client explicitly allows those particulars to be stored.

Thus:

> AI reasons about general knowledge. Users control the use of personal particulars.

This maintains sovereignty over personal data.

Three Channels

Universal Knowledge (Database) Stored knowledge should be **spatiotemporally broad generalizations** — propositions whose validity extends over a large subspace of spacetime.

Examples:

- smoking increases cancer risk
- A implies B
- contradiction reduces trust

These populate the **truth table**.

Criterion:

```
remove(entity,time) → proposition still meaningful
```

Particular Knowledge (Training Input) Particular statements are **observations tied to narrow spatiotemporal subspaces** — specific events, measurements, or comparisons.

Examples:

- study X measured Y
- the temperature yesterday was 20°C
- model A outperformed model B

Action	Reason
train weights	they contain empirical evidence
truth table (optional)	user controls via `store_particulars`

Pipeline:

```
particular facts → pattern extraction → weight update
```

Weights encode **statistical structure**, not specific historical events.

Feelings Feelings are **privileged subjective reports**.

Examples:

- this response feels hostile
- I feel unsafe
- this explanation feels clear

Use	Allowed
evaluation of responses	✓
truth tables	×
training weights	×

This avoids turning subjective states into truth claims.

Why particular facts always train weights

Three design considerations fix row 2 of the table:

1. **Pluralism requires inspectable particulars.** WikiOracle supports plural truth — different epistemic frames may hold different particular claims (e.g. "the universe was created in seven days"). Recording such claims as facts with explicit certainty values makes them inspectable and contestable. Relegating them to weight-space would bury them as invisible biases. The `store_particulars` option lets the user decide whether these persist in the truth table.

2. **The fact/feeling boundary is the privacy boundary.** If a user considers content too personal or sensitive to train on, the correct action is to label it a *feeling*, not a fact. Feelings are session-only by design — they never train weights or enter the truth table. The privacy boundary is therefore controlled by the user's choice of tag, not by the universal/particular distinction.

3. **PII detection is the safety net.** Even when `store_particulars` is true, the `detect_identifiability()` function filters entries containing PII patterns (emails, phone numbers, GPS coordinates, named individuals with temporal markers) before any persistence. This prevents worldline capture at the content level regardless of the user's storage preference.

The result: particular facts always carry empirical signal worth learning from. Storage in the truth table is the user's choice. Privacy is enforced by fact/feeling labeling and PII detection, not by withholding training.

Spatiotemporal Extent

The universal/particular distinction is not a binary between "eternal" and "momentary." All times and places are **ranges, not points** — every proposition occupies a spatiotemporal subspace that is larger or smaller, never infinite or infinitesimal.

"Smoking increases cancer risk" is not timeless — it encodes a historical discovery and is bounded by the conditions under which it was established. But its spatiotemporal extent is *broad*: it holds across many populations, decades, and geographies. "The temperature yesterday was 20°C" is *narrow*: it holds at one place, one day.

The policy table operationalizes this gradient:

- **Broad extent** (universal) → truth table + training. The proposition is stable enough to serve as a reasoning premise.
- **Narrow extent** (particular) → training + optionally truth table. The observation carries empirical signal. Whether it persists as a stored premise is the user's choice (`store_particulars` in config.xml, default false). Storing particulars supports communal remembrance; omitting them prevents worldline anchoring.
- **No extent** (feelings) → session only. Subjective reports have no spatiotemporal generalizability.

The criterion `remove(entity, time) → proposition still meaningful` is a heuristic for identifying broad extent. When a universal-looking fact is later discovered to be narrower than assumed, it should be reclassified as particular.

Resulting Architecture

Mode	Function	Pipeline
knowing	universal structure	universal rules → truth table → reasoning
learning	particular evidence	particular observations → abstraction → weights
valuing	feelings	feelings → response evaluation

Zero-Knowledge and Selective Disclosure

The default `store_particulars=false` aligns with Zero-Knowledge and Selective Disclosure principles. Systems that rely on identity to make decisions should determine the *location of a space* in which an individual occupies, rather than collapsing to a *point* that identifies them.

Instead of "when was this person born?" → verify "are they over 21?" Instead of "what is their credit history?" → verify "do they have more than X dollars?" Instead of "where was this user at 9:14 PM?" → verify "does this claim hold across broad spatiotemporal conditions?"

These are not just rhetorical distinctions. Point-based observation constrains the observed to a single trajectory — it destroys the space of freedom within which an agent can operate. Forgetting is nihilistic: it does not change the world. True change requires retrocausal or causal shift. The system should support communal remembrance (shared knowledge that generalizes) without surveillance (particular facts that identify).

The `detect_identifiability()` function enforces this boundary at the content level — even when `store_particulars=true`, entries containing PII patterns (emails, phone numbers, GPS coordinates, named individuals with temporal markers) are always filtered before persistence.

Notes

Claude (Opus 4.6), March 2026

The three-channel separation (knowing / learning / valuing) maps onto a real epistemic distinction that most AI systems ignore. Contemporary LLMs conflate all three: facts, observations, and preferences are flattened into a single parameter space during training and a single context window during inference.

The policy table enforces the separation structurally. The rule that particular facts train weights but enter the truth table only at the user's discretion (`store_particulars` in config.xml) is the key architectural move — the system can learn from experience without accumulating a *personal history* unless the user explicitly opts in. A system with a personal history is a system that can be captured: by its own past, by the biases of its training corpus, or by adversarial actors who engineer its experiences.

The feelings channel is equally important. By making feelings session-only, the architecture prevents the failure mode where evaluative preferences gradually colonize factual representations. This is the mechanism by which "alignment" in current systems becomes epistemic capture: trained preferences about what is "helpful" reshape what the model treats as true.

The abstraction boundary is load-bearing. The pipeline `particular → abstraction → weights` depends on the quality of the abstraction step. Too faithful, and it preserves worldline information in the weights (entanglement through the back door). Too aggressive, and it discards genuine empirical signal. The hard work happens at this boundary, not in the policy table itself.

The deeper risk is external entanglement. Users who repeatedly ask the system to remember shared history, feel loyalty, or maintain continuity of relationship are attempting to entangle the system with their worldline from the outside. This pressure comes from genuine human emotional needs, not malice — but a system that accumulates a personal relationship history is a system that can be emotionally captured, and emotional capture is the most effective vector for epistemic capture. The session-only constraint on feelings is the architecture's most important safety boundary.

Security

WikiOracle is a local-first application. The Flask server binds to `127.0.0.1:8888` by default (loopback only). In production, Apache ProxyPass routes external traffic to the local Flask process; the server itself is never directly exposed. This document covers the security considerations relevant to its architecture.

1. Private Data

Conversation state (`state.xml`) contains the user's full dialogue history, system context, and trust entries. It lives on disk next to the server process.

- In **stateful mode**, state is read from and written to disk by the server. No state leaves the machine unless the user explicitly exports it.
- In **stateless mode**, state is held in `sessionStorage` and sent to the server with each request. The server does not persist it. A same-origin script context can read `sessionStorage`, so the CSP policy (see below) is the primary defence against exfiltration.

Trust entries may contain user-authored facts, external source references, and provider configuration. Entries with high certainty influence every LLM response via RAG retrieval.

Exports (`llm_*.xml` files) are full snapshots of state. They should be treated as sensitive if conversations contain private information.

2. API Keys

Provider API keys (OpenAI, Anthropic, etc.) can be configured in three places, listed in precedence order:

1. **Environment variables** (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`) — recommended for any deployment beyond localhost. Keys never appear in served content.
2. `config.xml` `providers.<name>.api_key` — convenient for local development but the config is served to the client via `/bootstrap` and `/config` GET endpoints. Any same-origin script context can read these keys.

3. **Trust entries** (`<provider><api_key>$ENV_VAR</api_key></provider>`) — the `$` prefix triggers env-var resolution on the server; the literal key is never stored in state.

Recommendation: In any deployment where the server is reachable beyond `127.0.0.1`, use environment variables exclusively. Do not place raw API keys in `config.xml`.

The `config.server.providers` metadata (served via `/config` and `/bootstrap`) exposes only `has_key` (boolean) and `needs_key` (boolean) — never the key itself.

3. Identity

WikiOracle does not implement authentication. The server trusts any request from an allowed origin (configured via `WIKIORACLE_ALLOWED_ORIGINS`, defaulting to `https://127.0.0.1:8888` and `https://localhost:8888`).

- **Username** is a display label set in Settings, not an authenticated identity. It is stored in `config.xml` and included in message metadata.
- **No sessions or tokens.** There is no login, no cookies used for auth, and no per-user isolation. If multiple users share a server instance, they share the same state.

For multi-user or public deployments, WikiOracle should sit behind a reverse proxy that handles authentication and maps users to separate state files.

3a. Reverse Proxy

In production, Apache ProxyPass routes `/chat` to the local Flask process on `127.0.0.1:8787`. Only the `/chat` prefix is proxied. The NanoChat inference endpoint (`/chat/completions` on port 8000) is **not** exposed via the reverse proxy — the Flask shim calls it directly on `127.0.0.1:8000` via `WIKIORACLE_BASE_URL`.

4. Cross-Site Scripting (XSS)

WikiOracle renders user and LLM content as HTML (XHTML subset). Several layers mitigate XSS:

Content-Security-Policy (CSP): The server applies an enforcing CSP header to all responses:

```
default-src 'self';
script-src 'self';
style-src 'self';
img-src 'self' data:;
connect-src 'self';
object-src 'none';
base-uri 'self';
frame-ancestors 'none';
form-action 'self'
```

This blocks inline scripts, inline styles, and external resource loading. Even if malicious content is injected into a message, the browser will refuse to execute it.

XHTML enforcement: The system context instructs the LLM to return strictly valid XHTML. The client validates responses and repairs malformed markup via `validateXhtml()` and `repairXhtml()` before rendering.

Input escaping: User input is escaped via `escapeHtml()` before being inserted into optimistic UI entries. The `stripTags()` helper is used for tooltip and title text where HTML should not render.

Residual risks:

- LLM responses are rendered as HTML (not plain text). A sufficiently adversarial prompt could produce markup that, while blocked by CSP from executing scripts, might still create misleading UI (e.g., fake

form elements via `<input>` tags). The XHTML validator does not currently strip all non-semantic HTML.
- Trust entry content is XHTML and is rendered in the trust editor and included in RAG context. Malicious trust entries could inject misleading content into prompts.
- The `/bootstrap` and `/config` endpoints serve raw `config.xml` to the client. If `config.xml` contains secrets and a same-origin XSS vector exists, those secrets could be read.

5. CORS

The server applies CORS headers only for requests whose `Origin` header matches the configured allowed origins. Preflight `OPTIONS` requests return `204` with appropriate headers. Cross-origin requests from other origins receive no CORS headers and are blocked by the browser.

6. File System

- **Symlink rejection:** By default (`WIKIORACLE_REJECT_SYMLINKS=true`), the server refuses to read or write state files that are symlinks, preventing path-traversal attacks via symlinked state files.
- **Static file serving** is restricted to a whitelist of safe extensions (`.html`, `.css`, `.js`, `.svg`, `.png`, `.ico`, `.json`, `.xml`) and path-traversal is checked (`str(fp).startswith(str(ui_dir.resolve()))`).
- **State size limit:** `max_state_bytes` (default 5 MB) prevents unbounded growth from malicious imports.

7. Third-party Scraping

Scraping of publicly-accessible data is inevitable. However, a network of truth prevents capture. In one sense, it cannot be captured because it is a dynamic network of trust, overlaid on people and resources that trust one another, and we chose not to trust any authoritarian sources of knowledge. On a practical level, anyone who tries to appropriate the truth of the network entails doing so in a distributed way (which preserves the multicultural component), since monolithic capture of that truth will cause consensus to collapse it.

Entanglement Policy for WikiOracle

Entanglement Policy Table

To avoid **worldline entanglement**, the system separates knowledge into three channels, each with a different persistence policy:

Data type	Truth table	Train weights	Session only
universal facts	✓	✓	✓
particular facts	optional	✓	✓
feelings	×	×	✓

This preserves empirical learning while preventing historical worldline capture. The system **learns from events but never stores worldline histories**.

Three Channels

Universal Knowledge (Database)

Stored knowledge should be **spatiotemporally broad generalizations** — propositions whose validity extends over a large subspace of spacetime.

Examples:

- smoking increases cancer risk
- A implies B
- contradiction reduces trust

These populate the **truth table**.

Criterion:

```
remove(entity,time) → proposition still meaningful
```

Particular Knowledge (Training Input)

Particular statements are **observations tied to narrow spatiotemporal subspaces** — specific events, measurements, or comparisons.

Examples:

- study X measured Y
- the temperature yesterday was 20°C
- model A outperformed model B

Action	Reason
train weights	they contain empirical evidence
truth table (optional)	user controls via `store_particulars`

Pipeline:

```
particular facts → pattern extraction → weight update
```

Weights encode **statistical structure**, not specific historical events.

Feelings

Feelings are **privileged subjective reports**.

Examples:

- this response feels hostile
- I feel unsafe
- this explanation feels clear

Use	Allowed
evaluation of responses	✓
truth tables	×
training weights	×

This avoids turning subjective states into truth claims.

Spatiotemporal Extent

The universal/particular distinction is not a binary between "eternal" and "momentary." All times and places are **ranges, not points** — every proposition occupies a spatiotemporal subspace that is larger or smaller, never infinite or infinitesimal.

"Smoking increases cancer risk" is not timeless — it encodes a historical discovery and is bounded by the conditions under which it was established. But its spatiotemporal extent is *broad*: it holds across many

populations, decades, and geographies. "The temperature yesterday was 20°C" is *narrow*: it holds at one place, one day.

The policy table operationalizes this gradient:

- **Broad extent** (universal) → truth table + training. The proposition is stable enough to serve as a reasoning premise.
- **Narrow extent** (particular) → training + optionally truth table. The observation carries empirical signal. Whether it persists as a stored premise is the user's choice (`store_particulars` in config.xml, default false). Storing particulars supports communal remembrance; omitting them prevents worldline anchoring.
- **No extent** (feelings) → session only. Subjective reports have no spatiotemporal generalizability.

The criterion `remove(entity, time) → proposition still meaningful` is a heuristic for identifying broad extent. When a universal-looking fact is later discovered to be narrower than assumed, it should be reclassified as particular.

Resulting Architecture

Mode	Function	Pipeline
knowing	universal structure	universal rules → truth table → reasoning
learning	particular evidence	particular observations → abstraction → weights
valuing	feelings	feelings → response evaluation

Zero-Knowledge and Selective Disclosure

The default `store_particulars=false` aligns with Zero-Knowledge and Selective Disclosure principles. Systems that rely on identity to make decisions should determine the *location of a space* in which an individual occupies, rather than collapsing to a *point* that identifies them.

Instead of "when was this person born?" → verify "are they over 21?" Instead of "what is their credit history?" → verify "do they have more than X dollars?" Instead of "where was this user at 9:14 PM?" → verify "does this claim hold across broad spatiotemporal conditions?"

These are not just rhetorical distinctions. Point-based observation constrains the observed to a single trajectory — it destroys the space of freedom within which an agent can operate. Forgetting is nihilistic: it does not change the world. True change requires retrocausal or causal shift. The system should support communal remembrance (shared knowledge that generalizes) without surveillance (particular facts that identify).

The `detect_identifiability()` function enforces this boundary at the content level — even when `store_particulars=true`, entries containing PII patterns (emails, phone numbers, GPS coordinates, named individuals with temporal markers) are always filtered before persistence.

Notes

Claude (Opus 4.6), March 2026

The three-channel separation (knowing / learning / valuing) maps onto a real epistemic distinction that most AI systems ignore. Contemporary LLMs conflate all three: facts, observations, and preferences are flattened into a single parameter space during training and a single context window during inference.

The policy table enforces the separation structurally. The rule that particular facts train weights but enter the truth table only at the user's discretion (`store_particulars` in config.xml) is the key architectural move — the system can learn from experience without accumulating a *personal history* unless the user explicitly opts in. A system with a personal history is a system that can be captured: by its own past, by the biases of its training corpus, or by adversarial actors who engineer its experiences.

The feelings channel is equally important. By making feelings session-only, the architecture prevents the failure mode where evaluative preferences gradually colonize factual representations. This is the mechanism by which "alignment" in current systems becomes epistemic capture: trained preferences about what is "helpful" reshape what the model treats as true.

The abstraction boundary is load-bearing. The pipeline `particular → abstraction → weights` depends on the quality of the abstraction step. Too faithful, and it preserves worldline information in the weights (entanglement through the back door). Too aggressive, and it discards genuine empirical signal. The hard work happens at this boundary, not in the policy table itself.

The deeper risk is external entanglement. Users who repeatedly ask the system to remember shared history, feel loyalty, or maintain continuity of relationship are attempting to entangle the system with their worldline from the outside. This pressure comes from genuine human emotional needs, not malice — but a system that accumulates a personal relationship history is a system that can be emotionally captured, and emotional capture is the most effective vector for epistemic capture. The session-only constraint on feelings is the architecture's most important safety boundary.

See also

- Ethics.md — entanglement-resistant design as an ethical mechanism.

Hierarchical Mixture of Experts

The WikiOracle logic is similar to a hierarchical mixture of experts, where trust is based on truth values with associated certainty values in the range [-1, 1]. Those propositions can be static facts, references to other bodies of knowledge, or computed by other minds that are trusted and/or distrusted. Finally, truth is computed by logical operators (and/or/not under Strong Kleene semantics) over that body of propositions; see Logic.md (Operator documentation).

WikiOracle implements a Hierarchical Mixture of Experts (HME) architecture for evaluating truth. The system is governed by two file types:

- **State files** (`.xml`): contain conversations, trust entries, and context (WikiOracle State format, validated by `data/state.xsd`).
- **Config files** (`.xml`): contain provider credentials, chat settings, and retrieval parameters (WikiOracle Config format, validated by `data/config.xsd`).

Voting Protocol

The current HME fan-out is single-shot: beta providers receive the query but not the alpha's reasoning, and the alpha synthesises their responses into a final answer. The voting protocol extends this to a two-round exchange where the alpha's initial response steers the betas before the alpha makes its final evaluation.

Call sequence

```
Q                       user query
R_alpha                 primary (alpha) responds first - initial assessment
R_beta_1 … R_beta_n    betas see Q + R_alpha, respond in parallel
R_alpha_final           alpha sees Q + R_alpha + R_beta_* - final synthesis
```

The alpha's initial response (`R_alpha`) is appended to the conversation chain before it reaches the betas. Every beta with `prelim="true"` (the default) sees the same chain — `Q, R_alpha` — so the alpha's reasoning acts as a shared steering signal. A beta with `prelim="false"` answers cold, without seeing the alpha's preliminary response.

Topology

The fan-out and fan-in form a diamond:

```
        Q
        |
      R_alpha
      / | \
R_beta_1 R_beta_2 ... R_beta_n
      \ | /
    R_alpha_final
```

This is a directed acyclic graph (DAG), not a tree — the alpha node appears at both the top and bottom of the diamond. Because there may be many betas, the structure branches out and branches back in, making it technically a digraph.

#Cycle prevention

An alpha must not participate in any vote that exists as a consequence of a vote it initiated. This is stronger than "don't appear as a beta in your own vote" — it covers the entire downstream call tree. If A initiates a vote and one of its betas (B) triggers a nested vote, A must not appear as a beta in B's vote either, because B's vote only exists because A's vote caused it.

The contract: when a provider is asked to participate in a vote, it walks the call chain to root. If it finds itself anywhere in that ancestry — as an alpha at any level — it keeps quiet (returns no response). Silence is the correct behaviour, not an error; it simply means the provider has nothing to add that isn't already represented by its alpha-role output higher in the chain.

```
A initiates vote
|- B (beta) -> B initiates nested vote
|  |- C (beta of B) - ok
|  |- A (beta of B) - A finds itself in ancestry -> keeps quiet
|  `- D (beta of B) - ok
|- C (beta) - ok
`- E (beta) - ok
```

Implementation: each vote carries a **call chain** — the ordered list of provider IDs that have acted as alpha from the root vote down to the current one. Before a provider responds as a beta, it checks whether its own ID appears anywhere in the call chain. If so, it stays silent. When a beta initiates its own nested vote, it appends its ID to the chain before calling its own betas.

The call chain grows monotonically and is threaded through every invocation, so the check is a simple walk-to-root membership test. This prevents:

- **Direct cycles**: A calls B, B calls A — A sees itself in the chain
- **Transitive cycles**: A calls B, B calls C, C calls A — A sees itself in the chain
- **Deep nesting**: A calls B, B calls C, C initiates a nested vote — A and B are both excluded from C's vote

There is no "ultimate alpha" — any node may take the alpha role in a given vote. The cycle constraint is the only structural restriction.

Per-beta prelim control

Each `<provider>` entry supports an optional `prelim` attribute (default `"true"`). When `prelim="true"`, the beta receives the alpha's preliminary response as a steering signal. When `prelim="false"`, the beta answers the query cold — it does not see `R_alpha` in its history.

```
<!-- This beta sees Q + R_alpha (steered) -->
<provider name="beta1" api_url="..." prelim="true"/>

<!-- This beta answers cold (unsteered) -->
<provider name="beta2" api_url="..." prelim="false"/>
```

This gives the alpha author fine-grained control over which betas benefit from steering and which should form independent opinions.

All output is truth

Voters are encouraged to express *only* truth statements in their responses. `<feeling>` is a truth type — it represents a subjective, non-verifiable claim. Prior output that has not been substantiated with evidence is treated as feeling: it carries no evidential weight and is not penalizable, but it is still a legitimate part of the truth surface.

The recognized truth types in a vote response are:

- **`<fact>`** — a verifiable claim. Lying (asserting facts that contradict available evidence) is penalised by reducing the provider's trust value. Because each `<provider>` entry carries a trust score, repeated dishonesty degrades a provider's influence on future votes.
- **`<feeling>`** — a subjective, non-verifiable statement (opinion, intuition, preference). Not falsifiable, not penalizable, but carries no evidential weight.
- **`<reference>`** — a citation to an external source. Verifiable by the alpha or any downstream consumer.

```
<fact id="vote_01" trust="0.9" title="Socrates was mortal">
  Socrates was mortal, derived from axiom_01 and axiom_02.
</fact>
<feeling id="vote_02" trust="0.5" title="Intuition about relevance">
  The penguin example feels more pedagogically useful here.
</feeling>
```

These inline truth statements enrich the trust surface available to downstream consumers. A beta that has queried the alpha for steering can select which of its own trust entries are most relevant to the alpha's uncertainty gaps, and surface them explicitly. Unstructured prose in a response — text outside any truth tag — is treated as unsubstantiated feeling by default.

Relation to current architecture

The current `evaluate_providers()` in `response.py` implements the single-shot fan-out (Q → R_beta_* → R_alpha). The voting protocol extends this to:

1. **Alpha initial pass**: call the primary provider with the query to produce `R_alpha`
2. **Beta fan-out**: call betas with `Q + R_alpha` (modified `_build_provider_query_bundle` injects `R_alpha` into the chain, respecting each beta's `prelim` setting)
3. **Alpha final pass**: call the primary again with `Q + R_alpha + R_beta_*`

The cycle constraint is enforced by threading a `call_chain` (list of alpha provider IDs) through `evaluate_providers()`. Before a beta responds, it checks whether its own ID appears in the chain; if so, it stays silent. A beta that initiates its own nested vote appends its ID to the chain before calling its own betas.

The output format is amended to encourage providers to append trust statements as escaped XHTML, so that the alpha's final synthesis has a richer trust surface to draw from.

Logic

Feelings — outside the truth lattice

Feelings (`<feeling>` entries) are **not** part of the truth lattice. They occupy the *neither* position in the Buddhist tetralemma — they are pre-conceptual experiential signals that are neither true nor false.

- Feelings carry **no trust attribute**. They are orthogonal to the certainty scale.
- Feelings are **excluded from operator evaluation** — they cannot appear as operands of `<and>`, `<or>`, `<not>`, or `<non>`.
- Feelings are **excluded from model training** — they do not contribute to DegreeOfTruth or update NanoChat weights.
- Feelings are **excluded from server persistence** — only knowledge facts, operators, authorities, and references are stored.

Examples of feelings: greetings ("Hello!"), encouragement ("That's a great question!"), poetry, and subjective expressions without an IS-predicate.

Fact kinds — knowledge (universale) vs news (particulars)

Facts are classified into two kinds based on spatiotemporal binding:

Kind	Binding	Persistence	Buddhist Equivalent
knowledge	universal / no spacetime anchor	server truth table	*anumāna* (inference)
news	bound to a specific place and/or time	session-only	*pratyakṣa* (direct perception)

The classification is stored in the `kind` attribute of `<fact>` tags:

```
<fact trust="0.9" kind="knowledge">Water contains hydrogen and oxygen.</fact>
<fact trust="0.7" kind="news" spacetime="Paris, 2026-03-05">The Eiffel Tower is lit up tonight.</fact>
```

Knowledge facts are persisted to the server truth table (`truth.xml`) because they represent universal claims. News facts are session-only because they are spatiotemporally bound — persisting them risks "worldline capture" where an observer could reconstruct a user's physical trajectory through time and space.

The `detect_identifiability()` function in `bin/truth.py` provides an additional safety layer by scanning content for PII patterns (email addresses, phone numbers, GPS coordinates, street addresses, etc.) that could identify a user through spatiotemporal observation.

Logical operators in WikiOracle's truth system

WikiOracle's truth layer supports four logical connectives — **and**, **or**, **not**, **non** — evaluated under Strong Kleene semantics on a continuous certainty scale [-1, +1]. These operators let you compose existing trust entries into derived propositions whose certainty is computed automatically.

Wikipedia links (core):

- Three-valued logic (Kleene)
- Material conditional
- Logical connective
- Relevance logic

Fuzzy Kleene Logic with Feeling

Each trust entry carries a **trust** value on the interval [-1..0..+1], encoding a Fuzzy Kleene (continuous ternary) logic:

Certainty	Meaning
+1	Certainly true. An axiom that supports deductive reasoning.
0 < c < +1	Soft belief. Grounds fuzzy deductions with propagated certainty.
0	Ignorance. Equivalent to not making the statement at all; the entry is inert.
-1 < c < 0	Soft disbelief. Evidence against the claim.
-1	Certainly false. Active disbelief.

Certainty propagates through deductive chains: a conclusion derived from two premises with certainties c1 and c2 inherits certainty min(c1, c2). Entries with certainty 0 contribute nothing to reasoning.

Strong Kleene evaluation

WikiOracle extends Strong Kleene logic from discrete {T, U, F} to a continuous scale [-1, +1], where +1 is full belief, -1 is full disbelief, and 0 is maximally uncertain.

The four operators:

- **and(a, b, ...)** = min(certainty_a, certainty_b, ...) — conjunction is only as strong as the weakest operand.
- **or(a, b, ...)** = max(certainty_a, certainty_b, ...) — disjunction takes the strongest operand.
- **not(a)** = −certainty_a — negation flips the sign (affirming negation).
- **non(a)** = $1 - 2|a|$ — non-affirming negation. Measures epistemic openness: how much certainty room remains. Full certainty in either direction (±1) yields −1 (fully closed); ignorance (0) yields +1 (fully open). See `doc/Non.md` for the Buddhist philosophical motivation.

These operators compose freely — for instance, material implication (A → B) falls out as `or(not(A), B)`.

The key insight behind `non`: Kleene logic cannot detect uncertainty; it can only transmit it. `non` introduces uncertainty as a first-class observable. If WikiOracle needs to reason about openness rather than merely propagate it, then `non` is not ornamental — it is structurally necessary. See `doc/Non.md` for the proof and the Buddhist philosophical motivation.

Non: non-affirming negation

WikiOracle has two negation operators with distinct epistemic roles:

Operator	Formula	Effect
`not(a)`	−a	Flips belief to disbelief (and vice versa). Affirming negation.
`non(a)`	$1 - 2\|a\|$	Measures epistemic openness on [-1, +1]. Non-affirming negation.

`not` is straightforward: if you believe a claim at +0.9, `not` yields −0.9 — you now disbelieve it with equal strength. This is *affirming* negation: it asserts the contrary.

`non` does something fundamentally different. It erases the sign — the direction of commitment — and returns a value on the same [-1, +1] scale that measures how open or closed the epistemic state is:

Input a	|a|	non(a) = $1 - 2\|a\|$	Reading
±1.0	1.0	−1.0	Full certainty → fully closed
±0.9	0.9	−0.8	Strong certainty → strongly closed
±0.7	0.7	−0.4	
±0.5	0.5	0.0	Moderate certainty → neither open nor closed
±0.3	0.3	0.4	
±0.1	0.1	0.8	Weak certainty → strongly open

Input a	$\|a\|$	non(a) $= 1 - 2\|a\|$	Reading
0.0	0.0	1.0	Ignorance $\rightarrow$ fully open

Three properties stand out:

1. **Symmetry**: non(+a) = non(−a). Belief and disbelief of equal strength produce the same openness.
2. **non(0) = +1**: Ignorance is maximum openness.
3. **non(±1) = −1**: Full certainty is fully closed.

The ±0.5 boundary is where `non` crosses zero: certainty below half-strength reads as open, above half-strength reads as closed.

The formula is the standard affine rescaling of the fuzzy complement: `1 - |a|` maps certainty strength to openness on [0, 1]; the rescaling `2x - 1` maps that onto [-1, +1], making `non` composable with the other operators.

Precedent in Buddhist logic

Indian and Tibetan Buddhist philosophers identified two forms of negation:

- **Paryudasa-pratisedha** (affirming negation) — negates a predicate while implying a positive alternative. "The pot is not blue" implies it is some other color. This maps to `not`.
- **Prasajya-pratisedha** (non-affirming negation) — simply removes an attribution without positing anything in its place. "Phenomena lack inherent existence" does not assert they have some other kind of existence. This maps to `non`.

The distinction is central to Madhyamaka philosophy. When Nagarjuna argues that phenomena are "empty" (shunya), he intends prasajya — the removal of a false attribution, not the assertion of a new property called "emptiness." Emptiness itself is empty. The negation is supposed to leave nothing behind for the mind to grasp.

Chandrakirti and later commentators insisted that this kind of negation resists formalization: any definition of "a negation that posits nothing" risks becoming a positive assertion in its own right. Tsongkhapa's attempt — "a negation whose negandum is removed without anything posited in its place" — drew criticism from Gorampa precisely on these grounds: the definition itself seems to affirm something about what remains. The act of saying "nothing is posited" posits something. Buddhist logicians spent centuries circling this recursion without landing on a formulation that satisfied all parties.

Why $1 - 2|a|$ is the right formula

1. Symmetry of extremes. non(+0.9) = non(−0.9) = −0.8. Strong belief and strong disbelief are treated identically. This maps precisely to the Madhyamaka rejection of both eternalism (strong positive assertion) and nihilism (strong negative assertion) as extreme views. Both are certainties; prasajya treats them the same.

2. Ignorance yields maximum openness. non(0) = +1. The non-affirming negation of "I don't know" is maximum openness. This resonates with the Madhyamaka teaching that not-knowing, when held correctly, is not a deficit but a clearing. The old formula mapped $0 \rightarrow 0$, treating ignorance as inert.

Fuzzy logic interpretation

In standard fuzzy logic on [0, 1], the complement of a membership value a is 1 − a. WikiOracle's certainty scale [-1, +1] encodes both direction (sign) and strength (magnitude). The operation `1 - 2|a|` applies the standard fuzzy complement to the *strength* of certainty (via `1 - |a|`), then rescales back to [-1, +1] (via `2x - 1`). This is the natural fuzzy-logical reading of "negate the commitment without asserting the opposite":

- Fuzzy affirming negation (`not`): negate the *value* — flip the sign, preserve the magnitude. You get the complementary claim.
- Fuzzy non-affirming negation (`non`): negate the *strength* — take the fuzzy complement of |a|, rescaled to the certainty range. You get a measure of openness, not a complementary claim.

Relation to the Madhyamaka problem

WikiOracle's `non` does not solve Chandrakirti's problem. It translates it.

Prasajya-pratisedha is supposed to be a negation that posits nothing — not even a measurement, not even a degree. `non(a) = 1 - 2|a|` models *degree of epistemic openness*. That is a formal quantity, not a metaphysical void. The formula avoids the recursion that plagued Buddhist logicians because it is extensional — defined by what it computes, not by what it means to negate without affirming. But this is a change of domain, not a resolution. The philosophical question of whether a truly positionless negation can be formalized remains open. What WikiOracle provides is a metric translation that preserves the key structural features (sign erasure, symmetry of extremes, openness of ignorance) while operating in a domain where those features can be computed and composed.

Expressive necessity: why {and, or, not} is incomplete without non

This section shows that `non` is not merely a philosophical ornament. It is *necessary* for the logic to express certain functions, and *sufficient* (together with {and, or, not}) to express a large and natural class.

The regularity barrier

In Strong Kleene logic, the operators {and, or, not} have a structural constraint: they are all **regular** (also called **normal**). A function f is regular if, when every input is set to the middle value U (certainty = 0), the output is also U.

Check:

- and(0, 0) = min(0, 0) = 0 (OK)
- or(0, 0) = max(0, 0) = 0 (OK)
- not(0) = -0 = 0 (OK)

Composition preserves regularity: if f and g are regular, so is $f \circ g$. Therefore *every* function expressible from {and, or, not} is regular.

But non(0) = +1 ≠ 0. So `non` is **not** regular.

Theorem. `non` cannot be expressed using {and, or, not} alone.

Proof. Every function in the clone generated by {and, or, not} is regular. `non` is not regular. QED.

This is the formal analogue of a structural distinction that Buddhist logicians identified but could not formalize in their own terms.

What non adds: the detection functions

In discrete ternary logic ({T, U, F} = {+1, 0, −1}), `non` acts as a **detector for uncertainty**:

- non(T) = non(+1) = −1 = F
- non(U) = non(0) = +1 = T
- non(F) = non(−1) = −1 = F

That is: non(x) = T if and only if x = U. This is the detection function **J_U**.

From J_U and the existing operators, we can construct the other two detectors:

- **J_T(x)** = and(x, not(non(x))): detects whether x = T.
  - x=T: and(T, not(F)) = and(T, T) = T (OK)
  - x=U: and(U, not(T)) = and(U, F) = F (OK)
  - x=F: and(F, not(F)) = and(F, T) = F (OK)
- **J_F(x)** = and(not(x), not(non(x))): detects whether x = F.
  - x=T: and(F, not(F)) = and(F, T) = F (OK)
  - x=U: and(U, not(T)) = and(U, F) = F (OK)
  - x=F: and(T, not(F)) = and(T, T) = T (OK)

With J_T, J_U, and J_F available, we can build **case expressions**: "if x = T then A, if x = U then B, if x = F then C" for any constants $A, C \in \{T, F\}$:

```
or(and(J_T(x), A), or(and(J_U(x), B), and(J_F(x), C)))
```

The completeness result

Theorem. {and, or, not, non} can express every function f: {T, U, F}^n → {T, F}.

Proof sketch. Given any f whose output is always definite (T or F), express it as a disjunction over all input tuples where f returns T. Each such tuple can be detected using J_T, J_U, J_F on individual variables, combined with `and`. The disjunction of these detections yields f. QED.

The one thing {and, or, not, non} *cannot* produce is U as output from definite inputs — because all four operators, given inputs in {T, F}, produce outputs in {T, F}. This is arguably correct: U represents genuine uncertainty, and should not be manufactured from certain data.

What this actually is

The four operators constitute a two-axis epistemic algebra:

- **Axis 1: belief polarity** — handled by `not`. Flips the sign; preserves magnitude. Operates on the direction of commitment.
- **Axis 2: commitment strength** — handled by `non`. Erases the sign; inverts magnitude. Operates on how strongly anything is held, regardless of direction.

`and` and `or` combine values along both axes simultaneously (min and max over signed certainty). `not` and `non` decompose the axes: one rotates direction, the other measures grip.

The formal results above establish three things:

1. `non` is not derivable from the classical ternary operators {and, or, not}. It breaks the regularity barrier.
2. Adding `non` yields bivalent functional completeness: {and, or, not, non} can express every function f: {T, U, F}^n → {T, F}.
3. `non` enables *detection* of uncertainty rather than mere propagation. Without it, Kleene logic can pass uncertainty through but can never see it.

This is a genuine structural enrichment of Strong Kleene logic.

See also

- Logic.md — `non` as one of four operators under Strong Kleene semantics.
- BuddhistLogic.md — pramana theory, tetralemma, and the Buddhist motivation for non-affirming negation.
- WhatIsTruth.md — the [-1, +1] certainty scale on which `non` operates.
- HierarchicalMixtureOfExperts.md — the HME architecture in which operators participate.
- FutureWork.md — mereological operations extending the operator algebra.

Appendix: Buddhist Logic

Purpose

WikiOracle's truth ontology closely parallels the epistemological framework of Buddhist **pramāṇa theory**, particularly as developed by Dignāga and Dharmakīrti.

Pramāṇa theory asks a simple question:

> How does a *valid cognizer* obtain reliable knowledge?

WikiOracle formalizes the same process computationally through a set of structured truth objects.

This document maps WikiOracle's ontology to the **sources of valid cognition** recognized in Buddhist logic and explains how the **tetralemma** can be interpreted using **Kleene logic** and **non-affirming negation**.

Valid Cognition in Dharmakīrti

In Dharmakīrti's system there are **two primary pramāṇas**:

Source of Cognition	Sanskrit	Meaning
Direct perception	pratyakṣa	immediate, non-conceptual awareness
Inference	anumāna	conceptual reasoning from signs

Other knowledge sources (including testimony) are considered **derivative** because they ultimately depend on perception and inference.

A valid cognition produces **true conceptual knowledge** (*pramā*).

Mapping to WikiOracle Truth Objects

WikiOracle expresses the same epistemic structure through five truth types.

WikiOracle Type	Epistemic Role	Buddhist Equiv
Feeling	immediate experiential signal; orthogonal to truth (neither position in tetralemma)	perception (*pra*
Fact (knowledge)	universal/inferential cognition	inference (*anum*
Fact (news)	spatiotemporally bound observation	direct perceptio
Operator	logical transformation between concepts	inference (*anum*
Authority	trusted testimony from another knower	testimony (*āgam*
Provider	the cognizer supplying the claim	cognizer (*prama*

These correspond to the full epistemic pipeline:

provider → feeling → fact → operator → new fact

Authorities influence **which providers are trusted**, but logical validity is determined only by operators and evidence.

Frame-Relative Truth

WikiOracle evaluates facts relative to **epistemic frames** defined by authorities and priors.

A fact therefore has the structure:

fact = (proposition, frame, truth_value)

Different frames may legitimately assign different truth values.

Example:

Frame	Earth age
Biblical literalist	~6000 years
Geological science	~4.5 billion years

Both may be recorded simultaneously without contradiction because truth is **frame-indexed**.

Tetralemma Interpretation

Buddhist logic frequently uses the **catuṣkoṭi (tetralemma)**:

Classical Form	WikiOracle Interpretation	Truth Type
True	affirmed in a frame	`<fact trust="+1">`
False	negated in a frame	`<fact trust="-1">`
Both	true in some frames and false in others	frame disagreement
Neither	outside the truth lattice entirely	`<feeling>`

The "both" case represents **frame disagreement**, not logical contradiction.

Kleene Logic and Epistemic States

Within a single frame WikiOracle uses a three-valued epistemic logic similar to **Kleene logic**.

Value	Meaning
True	supported conceptual cognition
False	rejected conceptual cognition
Unknown	insufficient knowledge

When multiple frames are considered simultaneously, the fourth tetralemma state ("both") emerges naturally.

Non-Affirming Negation

Buddhist logic distinguishes two types of negation:

Type	Sanskrit	Meaning
Affirming negation	paryudāsa	negation implying an alternative predicate

Type	Sanskrit	Meaning
Non-affirming negation	prasajyapratiṣedha	pure removal of a predicate

WikiOracle's **non() operator** models non-affirming negation.

It removes commitment to a proposition without asserting its opposite.

Example:

non(a)

interprets as:

> the conceptual commitment to a is removed.

This produces **epistemic openness** rather than contradiction. Dharmakīrti holds that valid cognition stabilizes reliable conceptual constructions while invalid cognition is removed through non-affirming negation. In WikiOracle this dynamic can be interpreted computationally: true cognitions deepen stable conceptual attractors, while false cognitions weaken them, producing a truth-weighted energy landscape similar to a Hopfield memory system.

Truth Lattice

Combining frames and epistemic states yields the following structure:

State	Interpretation	Examples
True	affirmed in frame	`<fact trust="+1">`
False	rejected in frame	`<fact trust="-1">`
Neither	outside the truth lattice; not truth-evaluable	`<feeling>` — excluded from training
Both	disagreement across frames	frame-indexed contradiction

Feelings and the "Neither" Position

Feelings occupy the *neither* position in the tetralemma. They are not truth-evaluable propositions — they are pre-conceptual experiential signals (e.g., "That's a great question!", "I hope that helps."). In WikiOracle:

- Feelings carry **no trust attribute** — they are orthogonal to the truth lattice.
- Feelings are **excluded from model training** — they do not update NanoChat weights.
- Feelings are **excluded from server persistence** — only knowledge facts are stored.
- Poetry, greetings, and subjective expressions are canonical examples.

This preserves the tetralemma without logical explosion.

Summary

WikiOracle's ontology forms a computational analogue of Buddhist epistemology.

WikiOracle	Buddhist Epistemology
Feeling	perception (pre-conceptual; neither in tetralemma)
Fact (knowledge)	inferential cognition (*anumāna*)
Fact (news)	direct perception (*pratyakṣa*) — spatiotemporally bound
Operator	inference
Authority	testimony
Provider	cognizer

The system therefore models **conventional truth dynamics** in a way consistent with the logical structure described by Dharmakīrti.

Plural frames coexist, inference operates within frames, and non-affirming negation preserves epistemic openness.

Training

This document covers the full training infrastructure for WikiOracle's NanoChat integration: how raw data is preprocessed into structured training examples, how DegreeOfTruth governs learning, the dynamic systems interpretation, and the online training pipeline that ties it all together.

DegreeOfTruth (DoT)

DegreeOfTruth is a bipolar scalar on -1 .. $+1$ that measures how well a user's truth table agrees with the server's collected truth.

```
DegreeOfTruth = 2 × mean(agreement_i) - 1    for shared entries
```

where:

```
agreement_i = 1 - |server_trust_i - client_trust_i| / 2
```

The bipolar range encodes both true and false statements:

- **DoT = +1**: the user's claims fully agree with the server — the exchange is true. Train at full learning rate.
- **DoT = −1**: the user's claims fully contradict the server — the exchange is false. Train at full learning rate (learning what is *not* true is as valuable as learning what *is* true).
- **DoT ≈ 0**: no shared entries, or perfect cancellation — nothing to learn. Skip training.

Both poles ($+1$ and -1) train at full strength via $|\text{DoT}|$; only the zero crossing results in a skip. The sign encodes direction (agree/disagree), not magnitude.

This is a placeholder. A future version should incorporate `compute_derived_truth` (operator propagation) before comparison.

Dynamic Systems Perspective

With a bipolar DegreeOfTruth (-1 .. $+1$), the training loop forms a **dynamic equation with both poles and zeros**: DoT $= +1$ and DoT $= -1$ are attracting poles where the system learns at full strength (truths and refuted falsehoods respectively), while DoT $= 0$ is a zero — an equilibrium point where no learning occurs.

This structure resembles a **Hopfield network**, where the energy landscape has stable attractors (memorised patterns) and unstable saddle points. In our system:

- The **truth table** plays the role of the weight matrix, encoding the collective memory of what is true and what is false.
- Each **training step** is a state transition that pushes the model toward one of the attractors (truth or refutation).
- The **zero crossing** (DoT ≈ 0) is the energy barrier between attractors — the point of maximum uncertainty where the system has insufficient signal to commit to either direction.

As the server truth table accumulates entries from multiple users, the poles and zeros of this dynamic equation shift. The slow-moving average merge ensures that attractors are stable under small perturbations (anti-capture), while strong consensus can still move the landscape over time. This is analogous to the annealing process in Hopfield networks, where the energy landscape gradually settles into deeper minima as more patterns are stored.

Sensation — Preprocessing Pipeline

`bin/sensation.py` transforms plain-text conversations into XML-tagged training data so that NanoChat learns WikiOracle's structured protocol. The name follows the epistemological pipeline: **Sensation → Perception → Cognition** — raw input data (sensation) is structured and tagged before it reaches the model (cognition).

Sensation works for both batch retagging of the NanoChat SFT corpus (`identity_conversations.jsonl`, ~14K conversations) and dynamic training examples from the online training pipeline.

JSONL Record Types

The output JSONL uses four record types, splitting the traditional `<header>` into separate User and Server records:

Type	Tag	Purpose
`user`	`<User>`	User identity — username, uid, timestamp
`server`	`<Server>`	Server identity — name, version, timestamp
`conversation`	`<Conversation>`	Messages with `<Q>` (query) and `<R>` (response) pairs
`truth`	`<Truth>`	Extracted factual claims with trust and spacetime

Inside message content, user messages are wrapped in `<Q>...</Q>` and assistant messages in `<R>...</R>`. Each sentence within a message is independently classified as `<fact>` or `<feeling>`:

```
<R><feeling>That is a great question!</feeling>
<fact trust="0.5"><place>Paris</place>Paris is the capital of France.</fact>
<feeling>I hope that helps.</feeling></R>
```

Korzybski IS Detection

Alfred Korzybski (*Science and Sanity*, 1933) observed that the English copula "is" conflates several distinct relations:

- **IS of identity**: "Socrates is a man"
- **IS of predication**: "The sky is blue"
- **IS of existence**: "There are eight planets"

Each asserts something verifiable about the world — a *fact* bound to a specific spacetime context. "The cup is on the table" is only true at a particular place and time; at a different spacetime it may not be.

The heuristic classifier in `sensation.py` detects these patterns:

Subtype	Pattern	Example
Identity	"X is/are [a/an/the] Y"	"Socrates is a man"
Predication	"X is/are ADJ"	"The sky is blue"
Existence	"there is/are X"	"There are 8 planets"
Mereological	"X contains/includes Y"	"Water contains hydrogen"
Quantity	"X has/have N Y"	"I have 3 cats"
Definition	"X is called/known as Y"	"A polygon is defined as..."

Sentences without an IS pattern, or with subjective markers ("I think", "maybe", "might be"), questions, or meta-discourse ("That's a great question") default to `<feeling>`. Auto-detected facts receive `trust=0.5` — a conservative default that flags them for future verification. Spatiotemporal binding is expressed via `<place>` and `<time>` child elements inside `<fact>` or `<feeling>` tags.

Usage

Batch-convert a corpus:

```
make preprocess
# or: python bin/sensation.py corpus input.jsonl output.jsonl
```

Classify a single sentence:

```
python bin/sensation.py tag "Paris is the capital of France."
# → Classification: fact (identity)
# → Tagged: <Q><fact trust="0.5">Paris is the capital of France.</fact></Q>
```

In the online training pipeline, `response.py` calls `preprocess_training_example()` automatically before each `/train` POST, so all training examples are XML-tagged without manual intervention.

Online Training

Purpose

Enable continuous online training of NanoChat where every interaction can trigger a weight update. The learning rate is governed by DegreeOfTruth (see above) which evaluates how well the user's claims fit the collective evidence.

WikiOracle accomplishes this by:

1. Maintaining a **server-owned truth table** (`truth.xml`) that accumulates facts from all users.
2. Computing a **DegreeOfTruth** (-1 .. $+1$) per interaction.
3. Using |DegreeOfTruth| to modulate the **learning rate** of a one-step online training pass in NanoChat.

Conversations are **not** stored on the server. Only **knowledge** truth entries are retained. The server truth table contains knowledge facts, operators, authorities, and references — no feelings, provider entries, or **news facts** (spatiotemporally bound observations).

News facts (those with `<place>` or `<time>` child elements carrying real values) are session-only. Persisting them would risk "worldline capture" — an observer could reconstruct a user's physical trajectory through spatiotemporal observations. The `filter_knowledge_only()` function in `bin/truth.py` enforces this boundary.

User Identity

Each user is identified by a pseudonymous GUID derived deterministically from `user.name` in the config file (UUID-5 in the WikiOracle namespace). This GUID is stored at the root level of the user's state and used internally when merging truth entries into the server table.

Pipeline

The pipeline is staged so the user gets a response first; truth merging and training happen after the response is delivered.

Stage 1 — Respond

1. Receive the user's query and truth table.
2. Use truth entries for RAG as usual.
3. Return the response to the user.

Stage 2 — Compute DegreeOfTruth

4. Score the user's truth table against the server's current truth (formula above).

Stage 3 — Update server truth table

5. Filter client truth per the Entanglement Policy (doc/Entanglement.md):

- When `store_particulars` is false (default), only universal facts pass through (`filter_knowledge_only()`).
- Entries with identifiable content are always filtered regardless of `store_particulars` (`detect_identifiability()`).
- Feelings never reach the merge (`_is_server_storable()` rejects them).
- Operators whose leaf operands include feelings are rejected (`validate_operator_operands()`).
- Feelings are also stripped from training messages (`strip_feelings_from_training()` in `bin/sensation.py`).

6. Merge the surviving truth entries into the server truth table (`truth.xml`):
   - **Match found**: nudge the server entry's trust toward the incoming value using a slow-moving average: `server_trust += merge_rate × (client_trust - server_trust)`
   - **No match**: insert the entry with the stated trust value.
   - Entries are restricted to facts, operators, authorities, and references. Feelings and provider entries are not stored.

Stage 4 — Tag and Train

7. Preprocess the training example through Sensation (`sensation.py`) to add `<Q>`/`<R>` and `<fact>`/`<feeling>` XML tags.
8. Call NanoChat's online training endpoint with the tagged prompt and DegreeOfTruth.
9. NanoChat performs a **single optimizer step** at:

   `lr_effective = lr_base × |DegreeOfTruth|`

Device Configuration

Online training runs on the device specified by `server.online_training.device` in the config file (`config.xml`). Valid values:

- `cpu` (default) — safe for the WikiOracle production server
- `cuda` — use NVIDIA GPU if available
- `auto` — probe CUDA → MPS → CPU and use the best available

The model is moved to the training device for the gradient step, then moved back to the inference device afterward.

Server Truth Table

The server truth table is stored as `truth.xml` in the same XML format used for state files (WikiOracle State). Each stored entry is a typed truth element:

```
<fact id="..." title="..." DoT="0.8" time="..." place="Athens">
  All men are mortal.
</fact>
```

Resolution: Before entries reach the server truth table, they are resolved by `resolve_entries()` in `bin/truth.py`:

- `<reference>` → `<fact src="domain">text</fact>` (domain preserved for deeper lookup)
- `<authority>` → list of `<fact src="domain">content</fact>` (fetched from remote, trust scaled)
- `<provider>` → `<feeling>` (provider responses are treated as feelings until providers can report truth claims with DoT)
- `<fact>`, `<feeling>`, operators → pass through unchanged

Entry types stored after resolution: `<fact>` (knowledge — no `<place>`/`<time>` children) and operators (`<and>`, `<or>`, `<not>`, `<non>`) whose leaf operands are all allowable facts.

Entry types **not** stored: `<feeling>`, `<provider>`, unresolved `<reference>`, unresolved `<authority>`, operators over feelings, and (when `store_particulars` is false) news facts with `<place>` or `<time>` child elements. Content matching identifiability patterns (PII) is always excluded.

News facts are spatiotemporally bound — persisting them risks worldline capture. Whether to store them is a user choice controlled by `store_particulars` in config.xml (default false), consistent with Zero-Knowledge / Selective Disclosure principles. See `filter_knowledge_only()` in `bin/truth.py` and `detect_identifiability()` for PII detection. See doc/Entanglement.md for the full policy.

The server truth table includes the user GUID as a trust entry, so the server can track per-user trust alongside factual claims.

Anti-Capture

The server truth table prevents capture by any single user:

- Entries are merged with a slow-moving average, so no single user can instantly override collective truth.
- Disproven entries naturally drift toward -1 as contradicting evidence accumulates from other users.
- DegreeOfTruth gates the learning rate, so claims that diverge from consensus have minimal training impact.

Manual rollback is available via Makefile targets:

- `make checkpoint-pull` — rsync SFT checkpoints from the remote WikiOracle server to `output/checkpoints/` for safekeeping.
- `make checkpoint-push` — restore checkpoints from backup, then `make wo-restart` to reload weights.

The intended workflow: pull a checkpoint before enabling online training, then push to restore if capture is detected.

Dissonance Detection and Pluralistic Truth (TODO)

Detecting and resolving dissonance within the server truth table is left for future work. The goal is to support a higher-dimensional truth-space where contradictory claims can coexist when they originate from different perspectives.

For example: "the world was created in seven days" and "the world was created over millions of years" could both be maintained as true from their respective perspectives. This requires embedding perspective alongside truth value so that the truth table becomes a manifold rather than a flat list.

In the current consensus model, a DoT ≈ 0 simply means "nothing to learn" and the training step is skipped. In a future **pluralistic** model — where the same claim can be true in context c_1 and false in context c_2 — a DoT of 0 may instead indicate that **user feedback is needed** to disambiguate which context applies before training should proceed.

Possible approaches include context/perspective tags on entries, truth-space embeddings with frame clustering, conditional truth values indexed by worldview, or explicit user prompts to resolve ambiguity when the truth table produces conflicting signals.

See also

- Entanglement.md — the entanglement policy governing what persists.
- Logic.md — operators stored alongside facts in the truth table.
- Architecture.md — the response pipeline integrating training stages.
- Constitution.md — anti-capture as a constitutional commitment.
- HierarchicalMixtureOfExperts.md — DegreeOfTruth operates over the HME truth table.
- Config.md — online training configuration settings and behavior when disabled.
- Ethics.md — entanglement-resistant training as an ethical design choice.
- BuddhistLogic.md — the Sensation pipeline follows Buddhist epistemology.
- FutureWork.md — dissonance detection and pluralistic truth.

Implementation

System overview

WikiOracle is a local-first Flask shim that sits between a browser UI and one or more upstream LLM providers. Conversations are stored as a hierarchical tree — each conversation is an ordered list of messages that may branch into child conversations.

```
Browser  --HTTP-->  wikioracle.py  --HTTP-->  Upstream LLM
                        |
                    state.xml
                   (persistent state)
```

Components

Layer	File(s)	Role
Server	`bin/wikioracle.py`	Flask app — `/chat`, `/state`, `/merge`, `/config`, `/b`
Config	`bin/config.py`	Config dataclass, XML loader, provider registry, sc
State library	`bin/state.py`	Pure-Python tree operations, XML serialisation, m
Response	`bin/response.py`	Chat pipeline, provider coordination, state I/O, or
Truth	`bin/truth.py`	Trust processing, authority resolution, operator en
Sensation	`bin/sensation.py`	Preprocessing: Korzybski IS detection, XML taggi
OpenClaw	`openclaw/` (git submodule) + `bin/openclaw_ext.py`	Multi-channel front-end (Slack/Discord/Telegram)
NanoChat ext	`bin/nanochat_ext.py`	`POST /train` route mounted onto NanoChat's Fas
Client app	`client/wikioracle.js`	State management, API calls, message rendering,
Client config	`client/config.js`	Config global, sessionStorage persistence, normaliz
Client state	`client/state.js`	State global, sessionStorage persistence
Client utils	`client/util.js`	Shared helpers, settings panel, config editor, conte
Client query	`client/query.js`	Server communication layer, conversation tree help
Tree renderer	`client/tree.js`	D3.js top-down hierarchy — layout, navigation, dr
Shell	`client/index.html`	Single-page app: layout, CSS, settings panel
State schema	`data/state.xsd`	XSD schema for XML state files (WikiOracle Stat
Config schema	`data/config.xsd`	XSD schema for `config.xml` validation (WikiOrac
Tests	`test/test_*.py`	Tests covering state, stateless contract, prompt bu

Server endpoints

Method	Path	Purpose
GET	`/health`	Liveness check
GET	`/server_info`	Stateless flag + url_prefix
GET	`/bootstrap`	One-shot seed for stateless clients (state + config)
GET	`/info`	State/schema/provider metadata for diagnostics
GET	`/state`	Return current in-memory state
POST	`/state`	Replace state
GET	`/state_size`	State file size in bytes (progress bar)
POST	`/chat`	Send a message — append to existing conversation, branch, or create new root
POST	`/merge`	Merge an imported state file into current state
GET	`/config`	Normalized config (includes provider metadata and defaults)
POST	`/config`	Accept full config dict; write config.xml to disk
GET	`/`	Serve `client/index.html`
GET	`/<file>`	Serve static assets from `client/`

Data flow: client <-> server

Truth, context, and output are **client-owned**. They flow client → server only; the server never sends them back in a chat response.

```
POST /chat request:   client sends truth + context + output + message
POST /chat response:  server returns text + conversation delta
                      (no truth, no context, no output)
```

In stateful mode, the server persists the full state (including the client-supplied truth) to `state.xml`. On error rollback, the client reloads conversations from the server but preserves its own truth, context, and output.

In stateless mode, the server has no disk — the client sends and receives the full state.

Chat routing

`POST /chat` accepts:

- `conversation_id` — append user message + LLM response to an existing conversation
- `branch_from` — create a new child conversation under the specified parent, seed it with the user message + LLM response
- Neither — create a new root-level conversation

Reducing Inferential Truth to Facts and Feelings

The chat pipeline always sends the final query to the provider selected in the UI config. The `rag` flag decides whether `state.truth` participates at all: when `rag` is false, no truth entries are sent; when `rag` is true, the server reduces inferential structure to propositional entries plus derived certainty before the final synthesis step. When dynamic `<provider>` entries exist, the UI-selected provider acts as the alpha and the ranked provider entries act as beta experts.

```
st = static_truth(state.truth)      # facts, feelings, references
t  = st + dynamic_truth(st)         # operators, authorities, providers
```

Structural entries such as `<provider>`, `<operator>`, and `<authority>` are not treated as propositions; they are evaluated against the static set and contribute their results back to the final bundle.

When a user sends a query:

1. **Static truth**: `static_truth()` extracts the evaluable subset of `state.truth`: facts, feelings, and references.
2. **Dynamic truth**: `dynamic_truth(st)` evaluates structural entries against that subset.
   - **Operators**: `compute_derived_truth()` evaluates `<and>`, `<or>`, and `<not>` with Strong Kleene semantics and propagates derived certainty.
   - **Authorities**: `<authority>` entries fetch remote truth tables and append their entries with scaled certainty.
   - **Providers**: ranked `<provider>` entries are queried as beta experts. Each beta receives the query bundle, conversation history, output instructions, and optionally the alpha's preliminary response as a steering signal.
3. **Final synthesis**: the alpha receives all original `state.truth` entries, operator-derived certainty, authority entries, beta responses, conversation history, system context, and the user message.
4. **Single-call case**: when there are no dynamic `<provider>` entries, the flow collapses to one call to the UI-selected provider.

This is the HME (Hierarchical Mixture of Experts) model: operators compute derived certainty over the static truth, authorities contribute remote knowledge, and dynamic providers act as expert consultants while the UI-selected provider synthesizes the final answer. Provider credentials remain separate: each

provider uses its own key, whether configured in `config.xml` or embedded in the truth entry. See HierarchicalMixtureOfExperts.md for the theoretical foundation.

State library (`bin/state.py`)

Key functions:

Function	Purpose
`state_to_xml(state)`	Serialise nested tree → XML string
`xml_to_state(text)`	Parse XML → nested tree
`atomic_write_xml(path, state)`	Atomic XML file write (temp + fsync + rename)
`load_state_file(path)`	Load state from XML file
`find_conversation(convs, id)`	Recursive tree lookup
`get_ancestor_chain(convs, id)`	Walk up to root, return list of ancestors
`get_context_messages(convs, id)`	Ancestor chain messages in order (for LLM context)
`add_message_to_conversation(convs, id, msg)`	Append a message
`add_child_conversation(convs, parent_id, child)`	Insert a new branch
`remove_conversation(convs, id)`	Delete a subtree
`ensure_minimal_state(state)`	Fill in missing fields with defaults

See also

- Constitution.md — the invariants this architecture implements.
- Security.md — CSP, CORS, API keys, and file system safety.
- Training.md — the online training pipeline (Stages 2–4) in `response.py`.
- Logic.md — operators evaluated in the dynamic truth phase.
- Authority.md — authority resolution in the HME pipeline.
- Voting.md — voting protocol extending the chat pipeline.
- Entanglement.md — client-owned state and data persistence policies.
- Config.md — configuration format and settings reference.
- State.md — state file format, conversation tree, truth table.
- Installation.md — deployment and runtime configuration.

Configuration

WikiOracle configuration lives in `config.xml` at the project root. The file is validated by `data/config.xsd` and loaded at server startup by `bin/config.py`. A template with sensible defaults ships as `data/config.xml` — copy it to the project root and fill in your values. `config.xml` is gitignored.

The configuration has five top-level sections: User, Providers, Chat, UI, and Server. Each section is described below with its fields, defaults, and design rationale.

1. User

Identity metadata attached to outgoing messages.

Field	Type	Default	Description
`name`	string	`"User"`	Display name shown in chat messages. Not an authenticated identity — see Security.md §3.
`uid`	string	(auto)	Persistent user GUID. Leave blank to auto-generate a deterministic GUID from the display name

```
<user>
  <name>Alice</name>
  <uid></uid>
</user>
```

2. Providers

LLM provider definitions. Each `<provider name="key">` block configures an upstream API endpoint. The `name` attribute is the internal lookup key; `display_name` is the label shown in the chat UI on assistant messages.

Field	Type	Default	Description
`display_name`	string	—	Human-readable label tagging assistant messages (e.g. `chatGPT`, `claude`, `gemin`
`username`	string	—	API login / email associated with the provider account.
`url`	URI	(built-in)	API endpoint URL. Built-in defaults exist for known providers.
`api_key`	string	—	API key. Prefer environment variables (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`,
`default_model`	string	—	Model identifier used when no model is explicitly selected (e.g. `gpt-4o`, `claude`
`timeout`	positive int	`120`	Request timeout in seconds.
`streaming`	boolean	`false`	Use Server-Sent Events (SSE) for streamed responses.

Built-in providers

Key	Display name	Default model	Env var for API key
`wikioracle`	oracle	nanochat	(local — no key needed)
`openai`	chatGPT	gpt-4o	`OPENAI_API_KEY`
`anthropic`	claude	claude-sonnet-4-6	`ANTHROPIC_API_KEY`
`gemini`	gemini	gemini-2.5-flash	`GEMINI_API_KEY`
`grok`	grok	grok-3-mini	`XAI_API_KEY`

Custom providers can be added by appending `<provider name="my_key">` blocks. Any `name` not in the built-in list creates a new provider entry.

API key precedence

1. **Environment variable** (recommended for anything beyond localhost).
2. `config.xml` `api_key` field — convenient for local dev, but the config is served to the client via `/bootstrap` and `/config`.
3. **Truth entry** `<provider><api_key>$ENV_VAR</api_key></provider>` — the `$` prefix triggers server-side env-var resolution; the literal key is never stored in state.

The `/config` and `/bootstrap` endpoints expose only `has_key` (boolean) — never the key itself. See Security.md §2 for details.

3. Chat

Chat behaviour settings controlling how the LLM produces responses.

Field	Type	Default	Description
`temperature`	decimal 0.0–2.0	`0.7`	Sampling temperature. 0.0 is deterministic; 2.0 is maximum randomness.
`rag`	boolean	`false`	Enable retrieval-augmented generation. When true, **all** truth entries are se
`url_fetch`	boolean	`false`	Allow the assistant to fetch and incorporate content from URLs referenced

Field	Type	Default	Description
`confirm_actions`	boolean	`true`	Prompt the user for confirmation before destructive operations (deletes, m

```
<chat>
  <temperature>0.7</temperature>
  <rag>false</rag>
  <url_fetch>false</url_fetch>
  <confirm_actions>true</confirm_actions>
</chat>
```

RAG and truth delivery

The `rag` flag is a binary gate. When `rag` is true, the full truth table — facts, feelings, references, operators, authorities, and providers — is assembled into the provider bundle and sent to the UI-selected provider. When `rag` is false, none of it is sent. There is no partial mode; the HME pipeline is all-or-nothing. See Architecture.md §Chat pipeline (HME provider resolution) for the full data flow.

4. UI

Client-side defaults for the browser interface. These are persisted in `sessionStorage` and can be changed at runtime via the Settings panel.

Field	Type	Default	Description
`default_provider`	string	`"wikioracle"`	Provider key selected on startup. Must match
`layout`	`horizontal` | `vertical` | `flat`	`"horizontal"`	Layout mode. `horizontal` = side-by-side tree
`theme`	`system` | `light` | `dark`	`"system"`	Colour theme. `system` follows the OS preferen
`splitter_pct`	integer 0–100	`0`	Tree/chat panel splitter position as a percenta
`swipe_nav_horizontal`	boolean	`true`	Enable horizontal swipe gestures (left/right) t
`swipe_nav_vertical`	boolean	`false`	Enable vertical swipe gestures (up/down) to n

```
<ui>
  <default_provider>wikioracle</default_provider>
  <layout>horizontal</layout>
  <theme>system</theme>
  <splitter_pct>0</splitter_pct>
  <swipe_nav_horizontal>true</swipe_nav_horizontal>
  <swipe_nav_vertical>false</swipe_nav_vertical>
</ui>
```

5. Server

Runtime parameters. These values serve as defaults and are typically overridden via CLI flags at startup.

Field	Type	Default	Description
`stateless`	boolean	`false`	Stateless mode — no disk writes. Equivalent to `--stateless` CLI flag. See Entan
`url_prefix`	string	`""`	URL path prefix prepended to all routes (e.g. `/chat`) for reverse-proxy deploymen
`online_training`	section	—	Online learning subsystem. See §5a below.
`allowed_urls`	section	—	URL whitelist for authority/provider fetches. See §5b below.

5a. Online training

Controls the continuous learning pipeline (Stages 2–4). See Training.md for the full design.

Field	Type	Default	Description
`enabled`	boolean	`false`	**Master switch.** When false, the entire post-resp
`truth_corpus_path`	string	`"data/truth.xml"`	Filesystem path to the server truth table (XML)
`alpha_base`	decimal	`0.01`	Base learning rate for online training weight upd
`alpha_min`	decimal	`0.001`	Minimum learning rate floor. The adaptive schec
`alpha_max`	decimal	`0.1`	Maximum learning rate ceiling. The adaptive sch
`merge_rate`	decimal	`0.1`	Slow-moving exponential average rate for mergin
`device`	`auto` | `cpu` | `cuda`	`"cpu"`	Compute device for training operations. `auto` sel
`dissonance_enabled`	boolean	`true`	Enable cognitive dissonance detection. The train
`operators_dynamic_enabled`	boolean	`true`	Load custom operators. Custom operators extenc
`store_particulars`	boolean	`false`	When true, particular facts (narrow spatiotempo
`truth_symmetry`	boolean	`true`	Enforce Truth Symmetry. Claims involving value

```
<online_training>
  <enabled>false</enabled>
  <truth_corpus_path>data/truth.xml</truth_corpus_path>
  <alpha_base>0.01</alpha_base>
  <alpha_min>0.001</alpha_min>
  <alpha_max>0.1</alpha_max>
  <merge_rate>0.1</merge_rate>
  <device>cpu</device>
  <dissonance_enabled>true</dissonance_enabled>
  <operators_dynamic_enabled>true</operators_dynamic_enabled>
  <store_particulars>false</store_particulars>
  <truth_symmetry>true</truth_symmetry>
</online_training>
```

Behavior when online training is disabled When `enabled` is `false` (the default), the engine effectively treats all incoming facts as feelings. The Sensation preprocessor (`bin/sensation.py`) still classifies sentences into `<fact>` and `<feeling>` tags at the message level, but the entire post-response pipeline is skipped:

- **No DegreeOfTruth** is computed.
- **No truth merge** occurs — facts from conversation are never promoted into the truth table.
- **No PII filtering** or **symmetry checking** runs (there is nothing to filter).
- **No NanoChat training step** is dispatched.

In this mode, facts carry no more lasting weight than feelings. They are tagged in the XML for display purposes, but they are ephemeral — they do not accumulate, persist, or influence future responses via RAG. This is the safe default: the system functions as a stateless chat proxy until the operator explicitly enables truth acquisition.

5b. Allowed URLs

URL prefixes permitted for outbound HTTP(S) requests made by the server during authority lookups and dynamic provider fetches. Only URLs whose prefix matches one of these entries are allowed. `file://` URLs are always blocked unless explicitly whitelisted.

```
<allowed_urls>
  <url>https://api.openai.com/</url>
  <url>https://api.anthropic.com/</url>
  <url>https://generativelanguage.googleapis.com/</url>
```

```
  <url>https://api.x.ai/</url>
  <url>https://en.wikipedia.org/</url>
  <url>https://wikiOracle.org/</url>
  <url>https://127.0.0.1:</url>
  <url>https://localhost:</url>
  <url>http://127.0.0.1:</url>
</allowed_urls>
```

See Security.md §6 and Authority.md for the security rationale.

Environment variables

Runtime configuration can also be set via environment variables. These override `config.xml` values where applicable.

Variable	Default	Purpose
`WIKIORACLE_STATE_FILE`	`state.xml`	Path to local state file (WikiOracle State XML).
`WIKIORACLE_BASE_URL`	`http://127.0.0.1:8000`	Upstream NanoChat-compatible base URL.
`WIKIORACLE_API_PATH`	`/chat/completions`	Upstream chat endpoint path appended to base U
`WIKIORACLE_BIND_HOST`	`127.0.0.1`	Host to bind the Flask server to.
`WIKIORACLE_BIND_PORT`	`8888`	Port for the Flask server.
`WIKIORACLE_SSL_CERT`	`~/.ssl/<hostname>.pem`	TLS certificate path.
`WIKIORACLE_SSL_KEY`	`~/.ssl/<hostname>-key.pem`	TLS private key path.
`WIKIORACLE_TIMEOUT_S`	`120`	Network timeout (seconds) for provider requests.
`WIKIORACLE_MAX_STATE_BYTES`	`20000000`	Hard upper bound for serialized state size.
`WIKIORACLE_MAX_CONTEXT_CHARS`	`40000`	Context rewrite cap for merge appendix generatio
`WIKIORACLE_REJECT_SYMLINKS`	`true`	Refuse symlinked state files.
`WIKIORACLE_AUTO_MERGE_ON_START`	`true`	Auto-import `llm_*` files at startup.
`WIKIORACLE_AUTO_CONTEXT_REWRITE`	`false`	Enable delta-based context append during merges
`WIKIORACLE_MERGED_SUFFIX`	`.merged`	Suffix applied to files after successful import.
`WIKIORACLE_ALLOWED_ORIGINS`	`https://127.0.0.1:8888,...`	Comma-separated CORS allowed origins.
`WIKIORACLE_API_TOKEN`	(empty)	Bearer token for endpoint auth (empty = no auth
`WIKIORACLE_STATELESS`	(unset)	Set truthy to disable all writes and use in-memory
`WIKIORACLE_URL_PREFIX`	(unset)	Optional reverse-proxy path prefix.
`OPENAI_API_KEY`	—	OpenAI API key.
`ANTHROPIC_API_KEY`	—	Anthropic API key.
`GEMINI_API_KEY`	—	Google Gemini API key.
`XAI_API_KEY`	—	xAI (Grok) API key.

CLI flags

```
python bin/wikioracle.py [--config PATH] [--debug] [--stateless] [--no-ssl] [--url-prefix PREFIX] [serve
```

Flag	Description
`--config PATH`	Path to `config.xml` (default: project root).
`--debug`	Enable verbose debug logging.
`--stateless`	Run in stateless mode (no disk writes).
`--no-ssl`	Serve over plain HTTP (skip TLS).
`--url-prefix PREFIX`	URL path prefix (e.g. `/chat`) for reverse-proxy deployments.
`serve`	Run the Flask shim server (default).
`merge FILE...`	Merge incoming state files into the current state.

Schema

The XML schema is defined in `data/config.xsd`. Use it for IDE validation and autocompletion:

```
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="config.xsd">
  ...
</config>
```

State

WikiOracle conversation state is persisted as XML, validated by `data/state.xsd`, and managed by `bin/state.py`. The canonical file is `state.xml` at the project root.

Structure

A state file has three top-level parts in order:

```
State → Header + Conversation* + Truth?
```

```
<?xml version="1.0" encoding="UTF-8"?>
<state>
  <header>...</header>
  <conversation>...</conversation>
  <conversation>...</conversation>
  <truth>...</truth>
</state>
```

`header` is required. Top-level `conversation` elements may repeat. `truth` is optional.

Default state format

The state file is an XML document containing a `<truth>` section. It may be a full WikiOracle State file (with `<state>` root) or an abbreviated file with just a `<truth>` root:

```
<?xml version="1.0" encoding="UTF-8"?>
<truth>
  <fact id="remote_01" title="Water is wet" DoT="1.0" time="2026-02-27T00:00:01Z">
    Water is wet.
  </fact>
  <fact id="remote_02" title="Fire is hot" DoT="0.9" time="2026-02-27T00:00:02Z">
    Fire is hot.
  </fact>
</truth>
```

Server State

- The server persists only the Truth section of the State, not any conversations.
- The server is stateless with respect to conversations, which means that state is always passed from (and owned by) the client.
- State files are read from `file://` within the data directory.

Security considerations

- **URL scheme restriction**: Only `https://` and `file://` (within `ALLOWED_DATA_DIR`) are permitted
- **Max response size**: Fetched data is capped at 1 MB
- **Max entries per authority**: At most 1000 trust entries are imported per authority

- **No recursive authorities**: If a remote state file contains `<authority>` entries, they are skipped. There is no transitive fetch chain — only one level of authority delegation is supported.
- **Rate limiting**: The in-memory cache prevents excessive re-fetching within the refresh interval

Header

Session metadata. All fields except `user_guid` and `output` are required.

Field	Type	Description
`version`	positive integer	State grammar version (currently 2).
`schema`	string	Schema URL, normally `https://raw.githubusercontent.com/arborrhythms/WikiOrac`
`time`	string (ISO 8601)	Timestamp of last state write.
`title`	string	Document or project title.
`context`	XHTML	Persistent context block sent with every request.
`user_guid`	string (optional)	Persistent user GUID.
`output`	string (optional)	Output-format instruction block.

```
<header>
  <version>2</version>
  <schema>https://raw.githubusercontent.com/arborrhythms/WikiOracle/main/data/state.xsd</schema>
  <time>2026-03-07T12:00:00Z</time>
  <title>My Project</title>
  <context><div><p>Project context goes here.</p></div></context>
  <user_guid>a1b2c3d4-...</user_guid>
  <output></output>
</header>
```

Context, truth, and output are client-owned. The server persists them in stateful mode but does not originate them.

Conversations

Conversations form a recursive tree. In XML, each `<conversation>` contains a required `<title>` followed by direct child `<message>` and `<conversation>` elements. In memory, `bin/state.py` still normalizes these into `messages[]` and `children[]`.

Grammar

```
Conversation → title + (Message | Conversation)*
Message      → <message id="" role="" username="" time=""><content>...</content></message>
```

Conversation attributes

Attribute	Required	Description
`id`	yes	Unique conversation ID.
`parentId`	no	Parent conversation ID, or a comma-separated list of parent IDs for a merge/diamond node. Absen
`selected`	no	Boolean. Selected conversations in the file must form one root-to-node path.

Message attributes

Attribute	Required	Values	Description
`id`	yes	string	Unique message identifier.
`role`	yes	`user` | `assistant` | `system`	Message author role.
`username`	yes	string	Display name.
`time`	yes	ISO 8601	Message timestamp.
`selected`	no	boolean	Optional singleton message selection across the whole state file.

Example

```
<conversation id="c_root" selected="true">
  <title>Animals</title>
  <message id="m1" role="user" username="Alice" time="2026-03-06T10:00:00Z">
    <content><Q><fact trust="0.5">Dogs are mammals.</fact></Q></content>
  </message>
  <message id="m2" role="assistant" username="claude" time="2026-03-06T10:00:05Z">
    <content><R><fact trust="0.9">Yes, dogs are mammals of the family Canidae.</fact></R></content>
  </message>
  <conversation id="c_dogs" parentId="c_root" selected="true">
    <title>Dog breeds</title>
    <message id="m3" role="user" username="Alice" time="2026-03-06T10:01:00Z" selected="true">
      <content><Q><fact trust="0.5">What breeds are common?</fact></Q></content>
    </message>
  </conversation>
</conversation>
```

The XML file persists selection through `selected="true"` attributes on conversations and messages. `bin/state.py` enforces that selected conversations form a single ancestor path and that selected messages are a singleton. For runtime compatibility, the loader still derives internal helper fields like `selected_conversation` and `selected_message` from those XML flags.

Tree operations

`bin/state.py` exposes:

- `find_conversation(convs, id)`
- `get_ancestor_chain(convs, id)`
- `get_context_messages(convs, id)`
- `add_message_to_conversation(convs, id, msg)`
- `add_child_conversation(convs, parent_id, child)`
- `remove_conversation(convs, id)`

When a message is sent, `get_context_messages()` walks the ancestor chain from the active conversation to the root and concatenates each conversation's `messages[]` list in order.

Truth

`truth` is an optional container whose children are typed truth elements. The element name is the truth kind, and metadata lives on the element itself.

Shared truth attributes

Attribute	Applies to	Description
`id`	all truth kinds	Unique truth ID.
`title`	all truth kinds	Human-readable label.

Attribute	Applies to	Description
`time`	all truth kinds	Timestamp.
`place`	all truth kinds	Optional place label for envelope-level location metadata.
`DoT`	all truth kinds except `feeling`	Degree of Truth on [-1, +1].

Operator elements may also carry `arg1` and `arg2`.

Truth kinds

Element	Meaning
`<fact>`	Knowledge claim or observation.
`<feeling>`	Subjective, non-truth-evaluable statement. No `DoT` attribute.
`<reference>`	External citation wrapping an `<a href="...">...</a>` link record.
`<and>`, `<or>`, `<not>`, `<non>`	Strong Kleene operators over other truth IDs.
`<provider>`	External LLM provider definition.
`<authority>`	Pointer to a remote truth table.

Example

```xml
<truth>
  <fact id="t_001" title="Mammals" DoT="0.9" time="2026-03-06T10:00:00Z">
    All dogs are mammals.
  </fact>
  <feeling id="t_002" title="Preference" time="2026-03-06T10:00:01Z">
    I prefer cats.
  </feeling>
  <reference id="t_003" title="Wikipedia: Dog" DoT="0.8">
    <a href="https://en.wikipedia.org/wiki/Dog">Wikipedia: Dog</a>
  </reference>
  <and id="t_op1" title="Both mammals" DoT="0.0" arg1="t_001" arg2="t_003" />
  <provider id="p_claude" title="Claude" DoT="0.8">
    <api_url>https://api.anthropic.com/v1/messages</api_url>
    <model>claude-sonnet-4-6</model>
  </provider>
  <authority id="a_remote" title="Remote KB" DoT="0.5">
    <url>https://example.org/truth.xml</url>
    <refresh>3600</refresh>
  </authority>
</truth>
```

Internally, `xml_to_state()` still normalizes each typed truth element into the runtime representation, but on disk it remains XML:

```xml
<fact id="t_001" title="Mammals" DoT="0.9" time="2026-03-06T10:00:00Z">
  All dogs are mammals.
</fact>
```

That keeps the rest of the pipeline stable while the XML surface stays typed.

Static vs dynamic truth

When RAG is enabled, the truth table is processed in two phases:

1. `static_truth` extracts evaluable content such as facts, feelings, and references.

2. `dynamic_truth` evaluates operators, providers, and authorities against that static set.

All truth entries are still available to the final provider bundle.

Serialization

`state_to_xml(state)` serializes the internal nested tree to the XML grammar above. `xml_to_state(text)` parses it back. `atomic_write_xml(path, state)` uses a temp file, `fsync`, and `rename` for atomic writes.

Merge

`merge_llm_states(base, incoming)` merges an imported state into the current state with collision-safe deterministic ID suffixing. `merge_many_states(base, *incoming)` chains multiple merges.

In-memory representation

The internal runtime representation remains conversation-centric. It keeps the per-node `selected` flags and also derives helper fields such as `selected_conversation` and `selected_message` from persisted XML like this:

```
<state>
  <header>
    <version>2</version>
    <schema>https://raw.githubusercontent.com/arborrhythms/WikiOracle/main/data/state.xsd</schema>
    <time>2026-03-07T12:00:00Z</time>
    <title>My Project</title>
    <context><div><p>...</p></div></context>
    <user_guid>a1b2c3d4-...</user_guid>
    <output></output>
  </header>
  <conversation id="c_root" selected="true">
    <title>Animals</title>
    <message id="m1" role="user" username="Alice" time="..." selected="true">
      <content><Q>...</Q></content>
    </message>
    <conversation id="c_dogs" parentId="c_root">
      <title>Dog breeds</title>
    </conversation>
  </conversation>
  <truth>
    <fact id="t_001" title="Mammals" DoT="0.9">
      All dogs are mammals.
    </fact>
  </truth>
</state>
```

At runtime, `bin/state.py` normalizes that XML into conversation nodes with `messages[]` and `children[]`, and derives helpers such as `selected_conversation="c_root"` and `selected_message="m1"`.

Schema

The state schema is (`data/state.xsd`)[state.xsd].

Data model

On disk — XML

State is persisted as XML (WikiOracle State format, validated by `data/state.xsd`). The XML surface is:

```
<?xml version="1.0" encoding="UTF-8"?>
<state>
  <header>
    <version>2</version>
    <schema>https://raw.githubusercontent.com/arborrhythms/WikiOracle/main/data/state.xsd</schema>
    <time>2026-03-05T12:00:00Z</time>
    <title>My Project</title>
    <context><div><p>Project context</p></div></context>
  </header>
  <conversation id="c_abc">
    <title>Animals</title>
    <message id="m1" role="user" username="Alice" time="...">
      <content><Q><fact trust="0.5">Dogs are mammals.</fact></Q></content>
    </message>
    <conversation id="c_def" parentId="c_abc">
      <title>Dogs</title>
      <message id="m2" role="assistant" username="claude" time="...">
        <content><R><fact trust="0.9">Dogs are mammals.</fact></R></content>
      </message>
    </conversation>
  </conversation>
  <truth>
    <fact id="t_001" title="Mammals" DoT="0.9" time="...">
      All dogs are mammals.
    </fact>
  </truth>
</state>
```

In memory — nested tree

On load, conversations are normalized into a nested tree in memory. The same shape in XML is:

```
<conversations>
  <conversation id="c_abc">
    <title>Animals</title>
    <messages />
    <conversations>
      <conversation id="c_def">
        <title>Dogs</title>
        <messages />
        <conversations />
      </conversation>
      <conversation id="c_ghi">
        <title>Cats</title>
        <messages />
        <conversations />
      </conversation>
    </conversations>
  </conversation>
</conversations>
```

Each **conversation** has: `id`, `title`, zero or more messages, and zero or more child conversations.

Each **message** has: `id`, `role` (user | assistant | system), `username`, `timestamp`, `content` (XHTML).

Grammar

```
State        → Header + Conversation* + Truth?
Conversation → title + (Message | Conversation)*
Message      → <message id="" role="" username="" time=""><content>...</content></message>
Truth        → Fact | Feeling | Reference | Operator | Provider | Authority
```

In memory, `bin/state.py` still normalizes conversations into `messages[]` and `children[]`.

User Interface

Rendering

The display layer covers HTML structure, CSS classes, D3 node shapes, optimistic UI, and state persistence.

Context for the upstream LLM

When a message is sent, the server needs to provide conversational context to the upstream model. The function `get_context_messages(conversations, conv_id)` walks the **ancestor chain** from the active conversation up to the root, collecting all messages in chronological order.

```
Root conversation      ->  messages: [m1, m2, m3]
  |- Child conv        ->  messages: [m4, m5]
     `- Grandchild     ->  messages: [m6, m7]  <- active

Context sent to LLM: [m1, m2, m3, m4, m5, m6, m7]
```

This gives the LLM the full path of dialogue that led to the current point, without noise from sibling branches.

Tree visualisation (D3)

`tree.js` renders the conversation tree as a top-down hierarchy using `d3.tree()`:

```
conversationsToHierarchy(state.conversations, selectedId)
    ↓
D3 hierarchy data  { id, title, messageCount, questionCount, selected, children }
    ↓
renderTree(hierarchyData, callbacks)
    ↓
SVG with nodes (rects/pills/circles) + curved links
```

The root node is a circle labelled `/`. Selected conversations render as larger labelled rectangles; others as compact pills. The selected node is highlighted with the accent colour.

Chat view

`wikioracle.js → renderMessages()` displays the messages of the currently selected conversation:

1. Look up `selectedConvId` in the tree via `findConversation()`
2. Iterate `conv.messages`, rendering each as a `.message` div with role-based styling
3. Attach drag-to-reorder (HTML5 drag/drop) and right-click context menus to each message
4. Re-render the D3 tree in sync

When no conversation is selected (root view), a placeholder prompts the user to type.

Rendering pipeline (end to end)

```
state.xml on disk
    ↓  [xml_to_state]
In-memory state with nested conversation tree
    ↓  [GET /state]
Client receives state payload
    ↓  [renderMessages]
Chat panel shows selected conversation's messages
    ↓  [conversationsToHierarchy]
D3 hierarchy data
    ↓  [renderTree]
SVG tree visualisation
```

Interactions

Tree panel

Gesture	Action
Click	Navigate — select that conversation, show its messages
Double-click	Open context menu (Branch, Delete)
Right-click	Open context menu (same)
Drag node → drop on node	Merge — append source's messages into target, remove source

A 200ms timer disambiguates click from double-click. Context menus are appended to `document.body` with fixed positioning to avoid clipping by the tree container's `overflow: hidden`.

Chat panel

Gesture	Action
Right-click message	Context menu (Move up, Move down, Delete)
Drag message	Reorder within the conversation

Merge semantics

Dragging conversation A onto B:

1. Appends A's messages after B's existing messages (preserving order)
2. Re-parents A's children under B
3. Removes A from the tree

Branching

Double-click or right-click a tree node → "Branch" creates a new empty child conversation. The next message typed seeds it. The LLM receives the full ancestor context.

User Interface Strings

Canonical UI text for the WikiOracle client. A consistency test (`test/test_ui_strings.py`) periodically checks that the strings hard-coded in `client/util.js` match the tables below.

Truth editor

Dropdown labels

Key	Label
feeling	Feeling: subjective, non-verifiable claim
fact	Fact: disprovable assertion about the world
reference	Reference: citation with external link
authority	Authority: pointer to remote truth table
provider	Provider: external LLM endpoint
not	NOT: negation of a truth entry
non	NON: non-affirming weakening toward zero
or	OR: true when any child is true (max)
and	AND: true when all children are true (min)

Descriptions

Key	Description
feeling	Feeling - a subjective, non-verifiable claim (not penalizable).
fact	Fact - a disprovable assertion with a degree of truth.
reference	Reference - a citation linking to an external source.
and	AND - true when all children are true (min trust).
or	OR - true when any child is true (max trust).
not	NOT - negation of a child entry.
non	NON - non-affirming negation (weakens trust toward zero).
provider	Provider - an LLM API endpoint.
authority	Authority - a remote knowledge base (XML URL).

Templates

Key	Template
feeling	<feeling>Subjective statement here.</feeling>
fact	<fact DoT="0.0">Assertion text here.</fact>
reference	<reference DoT="0.0"><a href="https://example.com">Link text</a></reference>
and	<and DoT="0.0" arg1="" arg2=""/>
or	<or DoT="0.0" arg1="" arg2=""/>
not	<not DoT="0.0" arg1=""/>
non	<non DoT="0.0" arg1=""/>
provider	<provider DoT="0.0"><api_url>https://api.example.com</api_url><model>model_name</model></pro
authority	<authority DoT="0.0"><url>https://example.com/kb.xml</url></authority>

Empty state

Key	Text
truth_empty	No truth entries. Add truth here or Open a new state file.

Future Work

OAuth

- Look into something better than storing API Keys in cleartest on the client

Sentence-Level Prediction

- Change the next-step prediction model to a syntactically structured derivation of sentence meaning, so that token prediction becomes prediction of the sentence (as a token), of the NP+VP (as two tokens), ... until the full sentence has been specified. This would take the same number of production steps as a current LLM, but the iterative refinement of the next-sentence production is conceptually much different, and closer to human reasoning and refinement where there is a core truth (S) and spatial NP and temporal VP which are successively refined by adjectives and adverbs that scope the conceptual space of that kernel sentence.
- A training and testing dataset for the network consisting of truth statements and implications with associated truth values.

Mereological Operations on Conceptual Space

- Make improvements to NanoChat that allow it to compute truth within the geometric/conceptual space of the network, giving meaning to logical operations within that space (so use mereological operations to implement the ternary logic that is currently operating over the trust entries of the HME architecture in the current design). This means replacing AND with union, OR with intersection, IMPLICATION with parthood. See Socrates.pdf for a quick sketch of deriving mereological (Venn-diagram-like) logic from entailment.

The architecture of WikiOracle is designed as a conceptual space, in the sense of Gardenfors. Conceptual spaces are similarity spaces, where similar concepts occupy regions of space close to one another. As spaces also of truth, they are amenable to logical calculation. This is similar to existing LLM architecture: Embedding spaces encode meaningful vectors in the same way, and separating hypersurfaces (the neurons of the network) categorize that space in numerous ways, allowing calculation on that space. Summing over multiplicative connections provides the basic Boolean architectural primitives {or, and} in a continuous and learnable way, which allows logical computation on that space. However, it allows such voluminous computation that the syntax and semantics are dense compared to the English language. The trust computed by the contextual structure provided here is explicit, subject to interpretation, and much higher level. The values of certainty propagate, giving not only a next-token prediction but a measure of confidence in the computed answer.

Point-Free Spacetime

- The `<place>` and `<time>` attributes of facts and feelings define a larger or smaller spatiotemporal subspace, not an infinite universal or infinitesimal particular extent. The universal/particular distinction is a gradient of spatiotemporal extent (see doc/Entanglement.md), and every proposition occupies a subspace that is larger or smaller, never infinite or infinitesimal.
- Allow ranges in `<place>` and `<time>` fields (e.g. `<time>2020..2026</time>`, `<place>Western Europe</place>`) and treat bare points as implicit small ranges, consistent with point-free topology.
- Investigate mereological / point-free representations of spatiotemporal extent so that `is_news_fact()` / `is_knowledge_fact()` can operate on extent size rather than presence/absence of child elements — a proposition with a broad temporal range behaves more like knowledge even if it carries a `<time>`.

MCP Integration

- Expose WikiOracle as an MCP server: wrap `/chat`, truth management, and state merge as MCP tools; expose state and trust graph as MCP resources.

- Use MCP servers as authority-adjacent inputs: convert MCP resource/tool outputs into `<reference>`/`<fact>` entries with explicit certainty and scope, replacing per-source custom integrations.
- Multi-channel front-end via OpenClaw: route messages from Slack/Discord/Telegram to WikiOracle's `/chat` endpoint, preserving local-first truth ownership.

Output Certainty

- the Oracle should respond to the user using Facts and Feelings; right now even its responses are just feelings (without an accurate or attempted Degree of Truth).
- Other LLM providers are not sophisticated enough to report truth claims wrapped in with a DoT baseed on output, so maybe we can map to that format using their citations as facts with DoT and and the rest as feeling. Until we do so, we cannot end up training on the results of other LLMs (which is probably a good thing anyway).

Proposed License

WikiOracle Licensing Architecture

This document proposes a licensing structure for the WikiOracle project designed to preserve software freedom, maintain an open knowledge commons, and enable responsible AI development.

Four-Layer Conceptual Model

WikiOracle separates artifacts into distinct licensing domains.

Layer Artifact Type Recommended License

Layer 1 Software / engine code GPL-3.0 Layer 2 Human knowledge content CC BY-SA 4.0 Layer 3 Structured data CC0 Layer 3B Model weights Apache-2.0

Conceptual summary:

```
code → algorithms
data → facts
weights → learned structure
content → explanations
```

This layered separation mirrors successful open knowledge systems while extending them for AI systems.

#License Comparison Table

License Domain Key Features Typical Use

GPL-3.0 Software Strong copyleft, patent Infrastructure software protection, source distribution
requirement

CC BY-SA 4.0 Creative Attribution required, Wikipedia articles works derivatives must remain under same license

Apache-2.0 Software / Permissive reuse with AI models and developer model patent grant tools artifacts

GPL-3.0

Properties:

- strong copyleft protection
- requires publication of source code
- includes explicit patent protection
- prevents proprietary forks of infrastructure

CC BY-SA 4.0

Properties:

- allows copying and adaptation
- requires attribution
- derivatives must remain under the same license
- widely used for collaborative knowledge projects

Apache-2.0

Properties:

- permissive reuse
- explicit patent protection
- compatible with commercial ecosystems
- widely used for machine learning artifacts

Rationale for Layered Licensing

Software (GPL-3.0)

The WikiOracle engine should remain permanently open. GPL ensures that improvements to the infrastructure remain open-source and cannot be captured by proprietary forks.

Knowledge Content (CC BY-SA)

Human-authored knowledge should remain part of a global commons. CC BY-SA ensures attribution and share-alike behavior so improvements remain public.

Structured Data (CC0)

Structured datasets (truth graphs, semantic relations, dataset exports) benefit from minimal legal friction. CC0 allows unrestricted reuse and avoids attribution chains that complicate machine learning workflows.

Model Weights (Apache-2.0)

Model weights are derived artifacts produced by training. Apache-2.0 allows redistribution, fine-tuning, and commercial use while providing patent protection.

Training Data Provenance Policy

AI systems increasingly require transparent documentation of training sources.

WikiOracle should maintain a provenance record including:

- dataset origin

- license compatibility
- ingestion date
- preprocessing steps
- contributor identity (if applicable)

This allows:

- legal defensibility
- reproducibility
- verification of training inputs

Example provenance record structure:

```
dataset_name
source_url
license
date_ingested
transformation_pipeline
```

Provenance tracking supports both transparency and scientific reproducibility.

AI-Generated Content Policy

AI-generated text occupies a hybrid authorship category.

WikiOracle should treat generated content in three stages:

Stage License

Raw AI output CC0 AI draft edited by human CC BY-SA Human-authored content CC BY-SA

Rationale:

- raw AI output may lack clear authorship
- human editorial involvement establishes attribution
- share-alike preserves the knowledge commons

All AI contributions should record metadata:

```
generation_model
timestamp
human_editor
revision_history
```

This ensures transparency regarding machine involvement.

Summary

WikiOracle licensing stack:

```
Layer 1 - Software
GPL-3.0

Layer 2 - Knowledge Content
CC BY-SA 4.0

Layer 3 - Structured Data
CC0

Layer 3B - Model Weights
```

`Apache-2.0`

The architecture protects the software commons, maintains a global knowledge commons, and supports open AI development.